

Framework for Robust and Scalable Text Watermarking

Anonymous ACL submission

Abstract

Protecting intellectual property (IP) of text such as articles and code is increasingly important, especially as sophisticated attacks become possible, such as paraphrasing by large language models (LLMs) or even unauthorized training of LLMs on copyrighted text to infringe such IP. However, existing text watermarking methods are not robust enough against such attacks nor scalable to millions of users for practical implementation. In this paper, we propose FROSTWORT, the first training-free framework for robust and scalable text watermarking applicable across multiple text types (e.g., articles, code) and languages supportable by LLMs, for general text and LLM data provenance. FROSTWORT comprises several key innovations, such as being the first to use LLM as paraphrasers for watermarking along with a novel combination of techniques that are surprisingly effective in achieving robust verifiability and scalability. We empirically demonstrate that FROSTWORT achieves significantly better scalability, robust verifiability, and computational efficiency compared to SOTA article-text watermarking methods, and also showed how it could be directly applied to the watermarking of code.

1 Introduction

Achieving robust text data provenance via watermarking, independent of its digital format, is an important open problem impacting a wide-ranging set of real-world challenges. Among these is the issue of intellectual property (IP) enforcement: Content creators of any text format (e.g., articles or code) could potentially combat plagiarism and unauthorized distribution by watermarking their works to prove **data ownership**. However, existing text watermarking methods have been unable to meet the challenging requirements of many practical problem settings. For example, directly adding digital metadata or invisible Unicode watermarks (Rizzo et al., 2019; Taleby Ahvanooy et al., 2019) may have limited impact in proving text data ownership in adversarial settings as they may be easily removed. Existing natural language watermarking (Qiang et al., 2023; Yoo et al., 2023; Taleby Ahvanooy et al., 2019) that adjusts the text itself to encode IDs are also lack robustness

to paraphrasing attacks and have limited scalability in terms of the number of supportable IDs.

Adding to the challenge is the growing prevalence of generative large language models (LLMs) that may be trained on copyrighted text without permission. To enforce IP rights, content creators would need to be able to do **data provenance for LLMs**, i.e., *prove whether their set of work had been used to train 3rd party black-box LLMs*. While there have been recent works tackling this problem (Abdelnabi and Fritz, 2021; Zhang et al., 2023), they largely require intervening in the training process of the LLMs. This is unrealistic in practice, as not all LLM service providers may be cooperative due to incentive misalignment, and adversaries may also use open-source LLMs.

Hence, it is natural to ask *whether it is possible to develop a practical, robust and scalable text watermarking framework for protecting IP against both plagiarism and unauthorized training of LLMs*. For example, the watermarks should persist regardless of whether the text has been paraphrased, converted into speech or handwritten text, or used in unauthorized LLM training (e.g., fine-tuning, in-context learning) to produce a derived output. The framework should also be general enough to tailor to a wide range of text formats (e.g., natural language or code), and be scalable (i.e., support millions of users with reasonable computational cost).

In this paper, we propose FROSTWORT, the first training-free framework for robust and scalable text watermarking applicable across multiple text types (e.g., articles, code) and languages supportable by LLMs, for general text and LLM data provenance. *Rather than viewing LLMs as just sources of IP infringement, we introduce the novel perspective of using LLMs’ capabilities to protect existing IP*. Though simple, our training-free framework comprises several key innovations such as being the first to use LLM as paraphrasers for watermarking along with a novel combination of techniques that are surprisingly effective in achieving robust verifiability, scalability, and data provenance for LLMs, *beating state-of-the-art text watermarking methods*. In summary, our contributions are as follows:

1. We introduced a formulation of the robust and scalable text watermarking problem setting and lay out a set of desiderata to be satisfied (Section 2).
2. To tackle the challenges arising from these desiderata, we proposed FROSTWORT comprising novel innovations, including: (a) effective use of LLM paraphrasers

to watermark existing text with IP to be protected (Section 3.1); (b) combination of vocab permutation and a new orthogonal watermarking perturbation method *in token space*, to achieve high scalability and robust verifiability while preserving fidelity (Section 3.3).

3. We conducted comprehensive empirical evaluations, demonstrating that FROSTWORT achieves significantly better scalability, robust verifiability, and computational efficiency compared to SOTA article-text watermarking methods (Section 4.1), while meeting the desiderata for a variety of applications, including for LLM data provenance of articles (Section 4.3). We also showed how FROSTWORT could be directly applied to the watermarking of programming code (Section 4.2).

2 Problem formulation and Desiderata

Consider M clients, each with unique watermark ID $\mu \in \mathbb{M}$ and textual data $T_o \in \mathbb{T}$ (e.g., articles or code) represented as token sequences $T_o = [w_1, \dots, w_N]$, where each token w_i is from an ordered vocab space $\mathbb{V} = \{v_1, \dots, v_{|\mathbb{V}|}\}$. We assume that T_o has semantic content c (e.g., the IP content) that is only determined by its tokens and fully represents the text’s value. Text formatting is irrelevant, especially as adversaries can strip all formatting, making those channels unusable for watermarking¹.

Watermarking: Client i uses a watermarking operator $\mathcal{W}(\mu_i, T_o) \rightarrow T_w^{(i)}$ to produce a text $T_w^{(i)}$ that contains watermark μ_i , preserves c , and can then be used/distributed freely.

Attacks: There are adversaries who aim to claim the IP in $T_w^{(i)}$ through attacks $\mathcal{A}(T_w^{(i)}) \rightarrow T_{\text{sus}}^{(i)}$ that generate their own text $T_{\text{sus}}^{(i)}$ without the watermark μ_i while preserving semantic content c . Adversaries do not know μ_i but are able to perform several classes of attacks:

- $\mathbb{A}_1$: alter $T_w^{(i)}$ with word addition/removal/substitutions;
- $\mathbb{A}_2$: translate and paraphrase $T_w^{(i)}$ with an LLM;
- $\mathbb{A}_3$: watermark $T_w^{(i)}$ again with a different watermark;
- $\mathbb{A}_4$: using $T_w^{(i)}$ with any LLM for in-context prompting;
- $\mathbb{A}_5$: using $T_w^{(i)}$ to fine-tune any LLM.

Verification: Client i can use a verification operator $\mathcal{V}(\mu_i, T_{\text{sus}})$ to generate a score q indicating the likelihood that T_{sus} contains μ_i . They can then use a setting-specific threshold $\bar{q}$ to classify T_{sus} as watermarked with μ_i if $q \geq \bar{q}$. The operator $\mathcal{V}$ should be quick and not assume access to T_o , as in practice client i may have a large set of T_w and would need to automate the application of $\mathcal{V}$ to scan through a large set of $\{T_{\text{sus}}\}$ to identify any plagiarism (further discussion in Appendix K).

Given the above, a suitable watermarking framework should satisfy the following desiderata:

1. Fidelity. The watermarked text T_w should be semantically similar to T_o , i.e., $\mathcal{S}(T_o, T_w) \geq s$, where

¹Attacks include converting text to audio or non-digital formats like written text, which removes format-based watermarks (e.g., homoglyphs and zero-width Unicode characters) (Rizzo et al., 2019) or digital metadata.

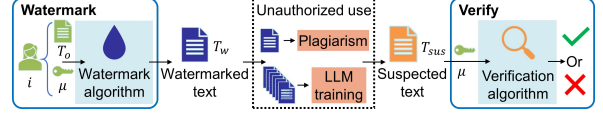


Figure 1: Schematics of problem formulation. Client i watermark text T_o with ID μ_i to watermarked text $T_w^{(i)}$. After manipulation by a third party, client can verify watermark in T_{sus} .

$\mathcal{S} : \mathbb{T} \times \mathbb{T} \rightarrow [0, 1]$ is a user-defined fidelity metric depending on the purpose and type of text (e.g., semantic similarity score for articles, or unit tests for code) and s is a setting-specific threshold. We define $\mathbb{T}_{c,s}^{\mathcal{W}} = \{T \in \mathbb{T} : \mathcal{S}(T_o, T) \geq s\}$ as the support set of all T_w that a watermarking operator $\mathcal{W}$ can possibly generate for T_o with content c under a s -fidelity setting.

2. Verifiability. The verification operator $\mathcal{V}(\mu_i, T_w^{(i)})$ should have high efficacy, accounting for Type I and II errors over various thresholds $\bar{q}$. We evaluate this with AUROC computed over a test set.

Note that there is a trade-off between fidelity and verifiability. Applying a stronger, more verifiable watermark tends to reduce text fidelity and the optimal setting depends on each use case. We can evaluate a watermarking scheme in general, while taking into account this trade-off, using its fidelity-verifiability Pareto frontier (e.g., as plotted in Figure 4a).

3. Robust verifiability. The verification operator on watermarked text after attacks $\mathcal{A} \in \mathbb{A}$, i.e., $\mathcal{V}(\mu_i, \mathcal{A}(T_w^{(i)}))$, retains high verifiability. This means that the watermark should remain verifiable even after attacks, which constrains framework design. For example, the verification operator should not extract μ in any subroutine, as an attacker may use it to get μ and devise an $\mathbb{A}_3$ attack to overwrite it (see Section 4.1).

4. Scalability. The framework should support a large $|\mathbb{M}|$ (set of IDs) while meeting all other desiderata.

3 Method

3.1 Increasing support set for watermarking via LLMs

We discuss three key insights to tackle challenges arising from these desiderata, before combining these to present our watermarking framework FROSTWORT (Framework for Robust and Scalable Text Watermarking of Original Text).

First, note that the **fidelity** desideratum is a major constraint to a scheme’s ability to meet the other desiderata. Intuitively, a scheme that can only generate a small set $\mathbb{T}_{c,s}^{\mathcal{W}}$ of possible watermarked text would have fewer ways to encode the watermark, leading to lower signal capacity (smaller $|\mathbb{M}|$, lower **scalability**), and less capacity for error correction to withstand attacks (lower **robust verifiability**).

For illustration, consider a basic semantic watermarking scheme (BASIC) that lists out synonyms for each

word in the original text T_o (e.g., big cat) and remembers a map of IDs to possible combinations of these synonyms (e.g., 01:big feline, 10:large cat, 11:large feline). Watermarking for ID μ is then selecting the text T_w with the matching synonym combination. Note that schemes like BASIC typically only have a relatively small support set $\mathbb{T}_{c,s}^W$ and hence limited watermarking possibilities.

However, LLMs can come up with many more possibilities and access a larger $\mathbb{T}_{c,s}^W$ compared to schemes like BASIC using mechanical paraphrasing rules (e.g., synonym replacement). Past works have shown that LLMs can effectively paraphrase text given suitable prompts (Shu et al., 2024; Witteveen et al., 2019). For example, while synonym replacement can only generate possibilities involving word replacements, an LLM may be able to completely reorder, break, or fuse sentences while preserving semantic content c . In general, as some expressions are more common, we can associate a probability distribution $p_c(T)$ over this set $\mathbb{T}_{c,s}^W$.

Intuitively, we can consider a suitable paraphrasing prompt combined with text T_o as tokens $\hat{c}$ that can constrain an LLM’s text generation to $\mathbb{T}_{c,s}^W$. Given $\hat{c}$, the LLM autoregressively access $p_c(T)$ by producing conditional probability distributions $p(w_j|\hat{w}_{1:j-1}, \hat{c})$ for token w_j at step j given the preceding sampled tokens $\hat{w}$, and sampling for each step until it deemed that it had conveyed c . Specifically, at step j , the LLM generates a vector of logits $L_j(\hat{w}_{1:j-1}, \hat{c}) \in \mathbb{R}^{|V|}$, where

$$p(w_j|\hat{w}_{1:j-1}, \hat{c}) = \text{softmax}(L_j(\hat{w}_{1:j-1}, \hat{c})). \quad (1)$$

We denote LLMs used this way as *LLM paraphrasers*. By using LLM paraphrasers, we significantly increase $\mathbb{T}_{c,s}^W$, which helps us better meet the fidelity, robust verifiability and scalability desiderata.

3.2 Increasing robustness using n -gram watermarking with LLM deviation correction

Given the extensive threat model, most watermarking schemes would face a major challenge in meeting the **robust verifiability** desideratum. For example, $\mathbb{A}_2$ paraphrasing attacks would likely break schemes such as BASIC which depend on word ordering², let alone attacks involving further processing by black-box LLMs (e.g., $\mathbb{A}_4$, $\mathbb{A}_5$ attacks). Instead, we could decompose $p_c(T)$ and the watermarked text T_w into multiple signal carriers, and embed the same watermarking signal to all. This way, we adopt a probabilistic approach where each carrier *could independently be used to verify a watermark*, to withstand attacks that can only corrupt a proportion of carriers.

Specifically, we could consider each consecutive n tokens in T_w as an n -gram carrier unit. At each LLM paraphraser token generation step j , we could apply a watermarking operator $\mathcal{W}$ (Section 3.3) that perturbs the logits of Equation (1) based on the

²Using example in Section 3.1, “large cat” → “cat that is large” would invert the embedded ID “10” to “01”.

ID μ and past $n - 1$ generated tokens: $\tilde{L}_j = \mathcal{W}[\mu, \hat{w}_{j-n+1:j-1}](L_j(\hat{w}_{1:j-1}, \hat{c}))$. The perturbed logits will cause a detectable bias in each n -gram, hence the more n -grams that persist after any attack, the higher the verifiability.

Meanwhile, in future generation steps j' , the *LLM paraphraser will correct deviations from semantic content c and preserve fidelity* given sufficient generation steps, as the subsequent logits $L_{j'}(\hat{w}_{1:j'-1}, \hat{c})$ are still conditioned on paraphrasing prompt $\hat{c}$.

This approach increases our framework’s robustness against not just paraphrasing attacks, but also more general LLM-based attacks (e.g., $\mathbb{A}_5$). Past works have shown that language models tend to generate few novel n -grams outside their training set for small n (McCoy et al., 2023). Hence, LLMs trained on text with our watermarked n -grams may more likely generate them in their output. Given sufficient queries to these LLMs, the watermark could then be reliably verified, which we empirically demonstrate in Section 4.

3.3 Increasing scalability with vocab permutation and orthogonal perturbation

Finally, we propose a watermarking operator $\mathcal{W}$ comprising two components: 1) vocab permutation, and 2) orthogonal perturbation. In this section, we will use a toy example (Vec) to show how these components work before presenting their general form. In Vec, we have logits $L = [3, 2, 1]$, indexed by an ordered set $V_o = \{\alpha, \beta, \gamma\}$ representing the token space, e.g., $L(\alpha) = 3$. Figure 2a presents L as a graph (V_o as x -axis).

Vocab permutation. The vocab permutation operator $\mathcal{P}$ produces a single permutation of V_o and L for any given key k_π (arrow ① in Figure 2a). The inverse operator $\mathcal{P}^{-1}$ reverses the permutation of $\mathcal{P}$ when provided the same key (arrow ② in Figure 2a). As $|V_o| = 3$, there are 6 possible permutations of L , plotted as graphs over a new ordered index $V_w = \{a, b, c\}$, which we can interpret as the watermarking space. Then, we define the average permutation operator $\bar{\mathcal{P}}$ acting on L (indexed by V_o) as one that takes a sequence of keys K_π , apply $\mathcal{P}$ to get L_{k_π} for each $k_\pi \in K_\pi$, and averages them to get a vector $\bar{L}$ (indexed by V_w). Note that when we use $\bar{\mathcal{P}}$ on L over all possible keys, we get a constant vector (e.g., $\bar{L} = \sum_{i=1}^6 L_i/6 = [2, 2, 2]$, ③ in Figure 2a).

Similarly, given a vector G indexed by V_w , which we can interpret as the watermark signal, the inverse operator $\mathcal{P}^{-1}$ permutes G and V_w given a key k_π , mapping it to V_o , the LLM-ordered token space (arrow ④ in Figure 2b). $\bar{\mathcal{P}}^{-1}$ acting on G analogously averages over all keys, and will also give a constant vector indexed over V_o (e.g., $\bar{G} = \sum_{i=1}^6 G_i/6 = [0, 0, 0]$, ⑥ in Figure 2b).

This leads to an interesting insight: the permutation operators provide a way for us to *add watermark signals to logits in a deterministically shifting V_w space (based on a sequence of keys) to boost verifiability and fidelity*. For illustration, assume that an LLM paraphraser produces L (in V_o -space) for all token generation steps. We use a long sequence K_π of pseudo-random uniformly

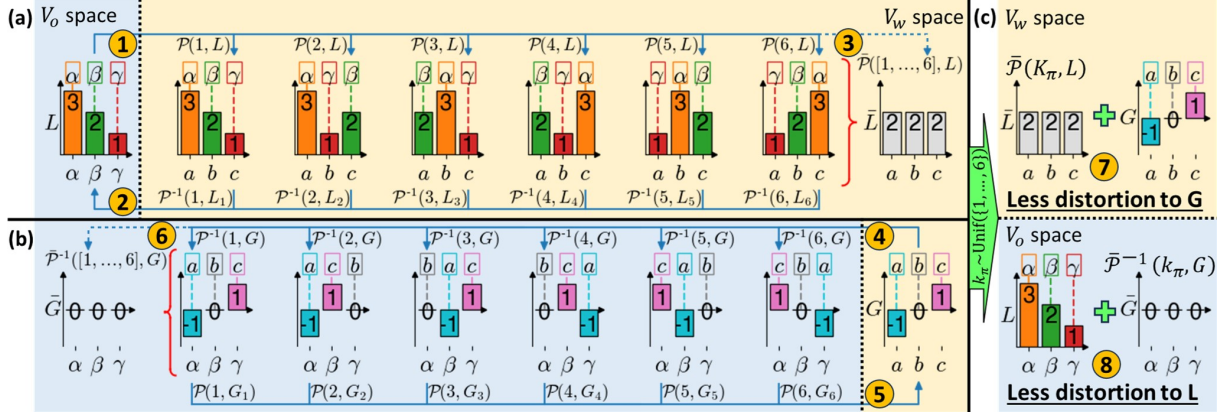


Figure 2: Intuition on permutation operators $\mathcal{P}$, $\mathcal{P}^{-1}$ applied on LLM logits L and watermarking signal G with toy example, Vec. (a) $\mathcal{P}$ applied to L in the V_o space results in 6 possible permutations in V_w space. This averages to constant vector $\bar{L}$. (b) Similarly, $\mathcal{P}^{-1}$ applied to G in V_w produces permutations in V_o . These averages to constant vector $\tilde{G}$. (c) With k_π sampled uniformly from the possible keys K_π over multiple LLM generation steps, $L + G$ in shows less distortion to G in V_w space, and to L in V_o space.

sampled keys to apply $\mathcal{P}$ on L multiple times (n -gram watermarking), and add the same watermarking signal G in each resulting V_w space for all instances. If we apply $\mathcal{P}^{-1}$ with K_π on the perturbed signal $L + G$, the distortion from the permuted L will effectively contribute only uniform background noise to G (⑦ in Figure 2c), which improves **verifiability**. If we instead convert $L + G$ back to V_o space (for token generation) with $\mathcal{P}^{-1}$ for all steps and apply $\bar{\mathcal{P}}$, we get the original logits with only uniform background noise from watermarking (⑧ in Figure 2c), which improves **fidelity**.

More generally, we define the vocab permutation operator $\mathcal{P}$ and its inverse $\mathcal{P}^{-1}$ as pseudorandom permutations over ordered sets V_o and V_w given a key $k_\pi \in \mathbb{K}_\pi$:

$$\begin{aligned} \mathcal{P}(k_\pi, V_o) &= V_o^{k_\pi} \\ \mathcal{P}^{-1}(k_\pi, V_w) &= V_w^{k_\pi} \\ \mathcal{P}^{-1}(k_\pi, \mathcal{P}(k_\pi, V_o)) &= V_o, \end{aligned} \quad (2)$$

where $V_o^{k_\pi}, V_w^{k_\pi}$ are uniform-randomly chosen permutations of V_o and V_w if k_π is sampled randomly. For a function L over V_o mapped to a vector of length $|V_o|$, we have $L(\mathcal{P}(k_\pi, V_o)) = L(V_o^{k_\pi})$ and we overload notation by defining $\mathcal{P}(k_\pi, L(\cdot)) \triangleq L(\mathcal{P}(k_\pi, \cdot)) = L_{k_\pi}$. As in the Vec example, $\mathcal{P}$ applied to a function (vector) can be viewed as the same function but with its domain permuted.

We then define an average operator $\bar{\mathcal{P}}$ over a sequence of keys K_π acting on a function L ,

$$\bar{\mathcal{P}}(K_\pi, L) \triangleq \frac{1}{|K_\pi|} \sum_{k_\pi \in K_\pi} \mathcal{P}(k_\pi, L), \quad (3)$$

which outputs an average function of L over V_w (denoted as $\bar{L}$). $\bar{\mathcal{P}}(K_\pi, L)$ will flatten towards a constant function over V_w for a sufficiently large K_π . To achieve this for our framework, we set $K_\pi = \{k_\pi \mid k_\pi = h_\pi(\mu, \hat{w}_{j-n+1:j-1})\}_j$, for all LLM paraphrasing steps j and where h_π is a hash function, which generates pseudorandom K_π sequences. Empirically, we clearly

observe the flattened and clear watermarking signals (see Figure 7 in Appendix).

Orthogonal perturbation: Our proposed perturbation operator $\mathcal{F}$ involves two sub-operations acting on V_w . It first maps each key $k_p \in \mathbb{K}_p$ to a unique function in a pre-defined family of orthogonal functions, and then adds the chosen perturbation function to the logits L_j of the LLM output in V_w space:

$$\mathcal{F}_1 : \mathbb{K}_p \hookrightarrow \{\phi : V_w \rightarrow \mathbb{R}^{|V_w|} \mid \langle \phi_i, \phi_l \rangle = \delta_{il}\} \quad (4)$$

$$\mathcal{F}(k_p, \kappa, L_j) = L_j + \kappa \mathcal{F}_1(k_p) \quad (5)$$

where $\langle \cdot, \cdot \rangle$ denotes the canonical dot product over V_w . Examples of orthogonal function families include the Fourier or square wave basis, discretized over $\mathbb{V}$. The key $k_p = h_p(\mu, z) \in \mathbb{K}_p$ is a client defined function h_p of ID μ , and also any metadata z (which could be extracted after verification as we demonstrate in Section 4.1) if required. κ is a scalar that controls the perturbation magnitude.

Combining both components, our watermarking operator (Figure 3, and Algorithm 1 in Appendix) for generation step j involves (a) using $k_\pi = h_\pi(\mu, \hat{w}_{i-n+1:i-1})$ and the permutation operator $\mathcal{P}(k_\pi, L_j)$ to transform logits from the V_o to V_w space, (b) applying the perturbation operator in Equation (5), and (c) transforming the perturbed logits back to V_o space using $\mathcal{P}^{-1}(k_\pi, \cdot)$ to produce a probability distribution for sampling and generation of the watermarked text T_w :

$$\begin{aligned} \tilde{L}_j &= \mathcal{W}(k_\pi, k_p, L_j) \\ &= \mathcal{P}^{-1}(k_\pi, \mathcal{F}(k_p, \kappa, \mathcal{P}(k_\pi, L_j))). \end{aligned} \quad (6)$$

Our verification operator will produce a score by computing the average cumulative token distribution of a text using $\bar{\mathcal{P}}(K_\pi, \cdot)$ and taking the inner product with $\mathcal{F}_1(k_p)$. Applying the right keys k_p and k_π on the suspected text T_{sus} will result in a high score q , else the score will be close to 0 (see Figure 3, and Algorithm 2

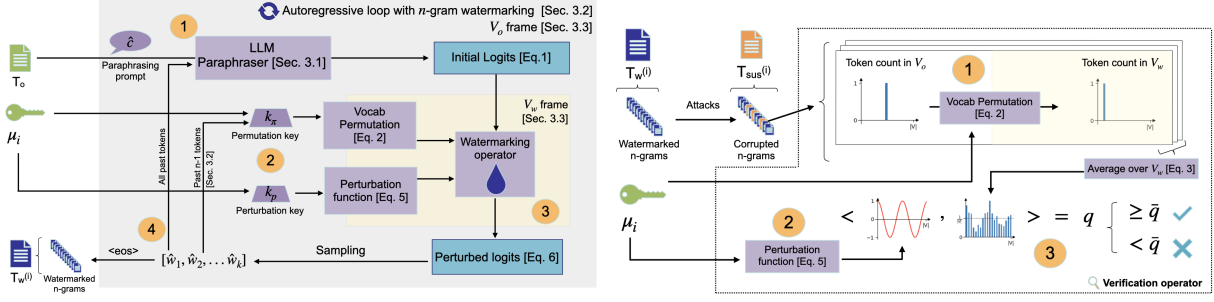


Figure 3: **Left:** Watermarking schematic. ① LLM paraphraser takes in T_o , produces initial logits. ② k_π and k_p from ID μ and metadata k_p for vocab permutation and perturbation function. ③ Perturb logits with Equation (6). ④ Sample perturbed logits, feed past tokens to the next iteration. **Right:** Verification schematic. ① Permute tokens from T_{sus} into V_w with μ and preceding $n - 1$ tokens, to get average cumulative distribution. ② Compute perturbation function $\mathcal{F}_1(k_p)$ linked to μ . ③ Compute verification score as inner product of $\mathcal{F}_1(k_p)$ and cumulative distribution, and compare with threshold.

in Appendix). Using orthogonal functions helps us improve verifiability by avoiding interference from other watermarks (e.g., added by adversaries as an $\mathbb{A}_3$ attack).

Notice that the many possible vocab permutations ($|\mathbb{V}|!$) and perturbation functions in any orthogonal function family $|\mathcal{F}_1|$ allows for a much large set of IDs compared to schemes like BASIC, helping with **scalability**. For example, up to $|\mathcal{F}_1| \cdot |\mathbb{V}|!$ IDs can be assigned to a unique permutation-perturbation function pair for watermarking. Using a relatively small $|\mathbb{V}| = 32000$ and the Fourier basis over that would yield a maximum $|\mathbb{M}| \sim 10^{130274}$. Schemes like BASIC only support M that scales with the number of possible synonym replacements for a given text.

In addition, with orthogonal functions, our framework also allows for the embedding of metadata during watermarking. For example, a client can use μ to verify that T_{sus} is watermarked, and also extract information on which article it was plagiarized from (Algorithm 3). We demonstrate this empirically in Section 4.1 using the Fourier basis as perturbation functions and Discrete Fourier Transform (DFT) for extraction.

3.4 FROSTWORD Framework

Our watermarking framework, FROSTWORD, combines these insights into a structured watermarking/verification process. For watermarking (Figure 3 left), given T_o and μ , FROSTWORD uses an LLM paraphraser to autoregressively paraphrase a text T_o , producing initial logits for the new text T_w [Step ①]. The ID μ is used to seed the vocab permutation operator (Equation (2)) for mapping the logits to V_w space, and chooses the perturbation function (Equation (5)) [Step ②], both of which will be used in the watermarking operation (Equation (6)) to produce the perturbed logits [Step ③]. The LLM samples the perturbed logits to produce a watermarked token, and for the next token loop, the past $n - 1$ tokens are used to seed vocab permutation while all past tokens are fed as context which helps the LLM paraphraser maintain T_w fidelity despite watermarking [Step ④].

For verification (Figure 3 right), each token in T_{sus}

is counted in V_w space as specified by μ and the previous tokens in the same n -gram unit, producing an average cumulative token distribution [Step ①]. The ID μ also specifies a specific perturbation function [Step ②], which is used to perform an inner product with the cumulative distribution to compute a verification score q [Step ③].

Practical considerations. FROSTWORD is highly adaptable, i.e., it can be implemented with different LLM as paraphrasers, allowing our framework to achieve better watermarking performance and support more text types as the LLM landscape evolves. Methods like prompt engineering (Wei et al., 2022; Lin et al., 2023) and Reflexion (Shinn et al., 2023; Madaan et al., 2023) may also help to boost performance in some settings, as we demonstrate in our code watermarking experiments (Appendix G.2). We elaborate further on possible large-scale deployment methods of FROSTWORD and other practical considerations in Appendix L.

4 Experiments

4.1 Data ownership

For watermarking of text articles, we demonstrate the effectiveness of FROSTWORD with experiments using text samples T_o from the c4 realnewslike dataset (Raffel et al., 2020), comprising articles with mean token length of 412. The experiments mirror realistic scenarios, for e.g., news outlets watermarking their articles before publishing them to be able to effectively scan the internet for, and verify, plagiarized content (Brewster et al., 2023). For this setting, we evaluate the semantic similarity $\mathcal{S}$ using the Semantic Textual Similarity (STS) score based on the all-mpnet-base-v2 model³ ($\mathcal{S}$ for sample text pairs are provided in Appendix E.5).

For benchmarks, we consider two recent linguistics-based watermarking methods: M-BIT by Yoo et al. (2023) and P-NLW by Qiang et al. (2023). These methods are advanced variants of BASIC that use deep

³<https://huggingface.co/sentence-transformers/all-mpnet-base-v2>

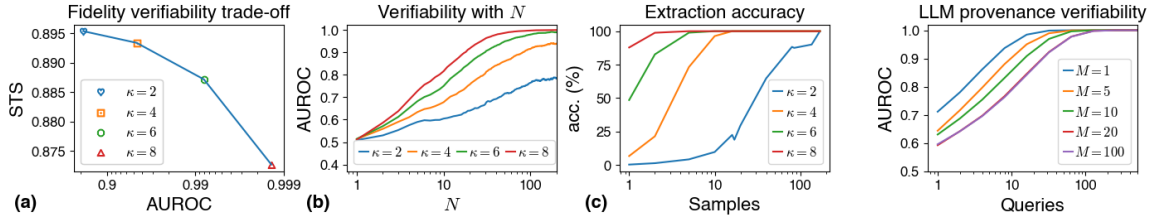


Figure 4: Higher watermarking strength κ improves verifiability and extraction accuracy. (a) Increasing κ trades off fidelity for higher verifiability. (b) Longer token length N improves verifiability. (c) Combining more pieces of text improves extraction accuracy towards 100%. Extraction accuracy is significantly higher than random guess accuracy of 0.003125%.

Figure 6: More queries improve LLM provenance verifiability. Increasing number of clients M in the training dataset only slightly decreases verifiability.

learning to improve watermarking performance (details in Appendix E.3). To implement FROSTWORT, we use llama-2-13b-hf⁴ as the paraphraser, and the Fourier basis for the perturbation functions. Additional details such as paraphrasing prompts are in Appendix E.

Fidelity-verifiability. We consider the fidelity and verifiability of the schemes before adversarial attacks. Verifiability is computed as the AUROC based on varying their respective classification thresholds, i.e., the verification score threshold $\bar{q}$ for FROSTWORT, and bit-error rate threshold for M-BIT and P-NLW.

FROSTWORT allows for adjustable watermarking strength, which clients can use to calibrate the fidelity-verifiability trade-off based on their use case. Figure 4a shows the Pareto frontier of the trade-off. Stronger watermark strength κ improves verifiability but also introduces larger distortions to the LLM paraphrasing process, decreasing the fidelity of watermarked text. For our experiments, we mainly used $\kappa = 6$, achieving a mean AUROC of 0.992 and STS of 0.887. Even for shorter texts of just 100 tokens (*about* 65 words), FROSTWORT achieves high verifiability with an AUROC of 0.98 (Figure 4b). Additional results are in Appendix E.4.

Note that M-BIT and P-NLW were designed with only one setting, allowing for only a single fidelity-verifiability score, with mean STS scores of 0.998 and 0.942 respectively, and corresponding AUROC scores of 0.987 and 0.882. While the STS scores are high, it is expected given that the schemes only make minor edits to T_0 which would be more fragile to attacks, as we will see later. Additionally, the word replacements by M-BIT and P-NLW introduced noticeable linguistic errors that are difficult to evaluate and not captured by the STS score (shown in Appendix E.5).

Robust verifiability. We consider the various classes of attacks $\mathbb{A}$ mentioned in Section 2. Details of the setup for each attack and additional results are in Appendix F.

$\mathbb{A}_1$ attacks are insertion, deletion, and synonym substitution attacks that are often considered in past works. As shown in Figure 5, robust verifiability of FROSTWORT shows only a very slight decrease even with

strong attacks on 20% of words, while that of benchmarks M-BIT and P-NLW fall drastically with increasing attack strength.

$\mathbb{A}_2$ involves translation and paraphrasing attacks, which are more realistic and effective attacks that can achieve higher fidelity and verification reduction than $\mathbb{A}_1$ and had not been considered by past text watermarking works. We perform **translation** attack to translate the watermarked text to Spanish and back to English, and **paraphrasing** attack to paraphrase the watermarked text. Again, the verifiability of FROSTWORT remains significantly higher than benchmarks post-attack.

$\mathbb{A}_3$ involves using the same scheme to try overwrite the existing watermark with another watermark. For FROSTWORT, the 1st watermark remain verifiable even after the 2nd is added, given the design of $\mathcal{P}$ and $\mathcal{F}$ with vocab permutation and orthogonal perturbation functions that minimizes interference of the 2nd watermark on the 1st. However, this attack destroys the verifiability of M-BIT and P-NLW, as the 2nd watermark process almost always chooses the same word positions as the original process, overwriting μ_1 . Furthermore, the benchmark schemes extracts μ_1 as part of verification, enabling targeted overlap watermark attacks which we demonstrated in Appendix F.3.

$\mathbb{A}_4$ uses T_w for in-context prompting of any LLM to perform tasks that rely on the IP or semantic content of T_w . For illustration, we considered the case where adversaries use an LLM to answer questions regarding watermarked articles. As this attack totally changed the structure of the texts, the watermarks of M-BIT and P-NLW were removed. However, with FROSTWORT, watermarks were still verifiable due to the preservation of watermarked n -grams from the context to the response.

$\mathbb{A}_5$ which involves using text containing IP for unauthorized LLM training such as fine-tuning will be discussed in Section 4.3.

Scalability. As mentioned in Section 3.3, FROSTWORT has a large maximum scalability of $M = |\mathcal{F}_1| \cdot |\mathcal{V}| \sim 10^{130274}$ based on our implementation using the Fourier perturbation function, and Llama-2 model as LLM paraphraser. In comparison, M-BIT and P-NLW, have scalability dependent on the number of possible synonym replacements in any given text, which is lim-

⁴<https://huggingface.co/meta-llama/Llama-2-7b-chat-hf>

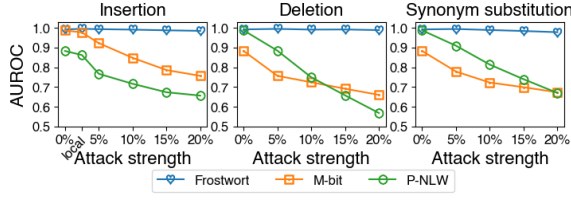


Figure 5: FROSTWORT demonstrates robust verifiability under $\mathbb{A}_1$ (insertion, deletion, and synonym substitution attacks) with minimal degradation in AUROC compared to M-BIT and P-NLW.

Table 1: Robust verifiability under attacks: translation $\mathbb{A}_{2-T}$, paraphrase $\mathbb{A}_{2-P}$, overlap watermark $\mathbb{A}_3$ and in-context prompting $\mathbb{A}_4$.

	Pre-attack	$\mathbb{A}_{2-T}$	$\mathbb{A}_{2-P}$	$\mathbb{A}_3$	$\mathbb{A}_4$
FROSTWORT	0.992	0.951	0.881	0.815	0.775
P-NLW	0.885	0.475	0.508	0.724	0.502
M-BIT	0.988	0.567	0.363	0.664	0.525

ited by text length and varies for different text. On the $\mathcal{C}_4$ dataset with a mean article length of 355 words, M-BIT and P-NLW can only embed a mean of 9.5 bits ($M \sim 10^3$) and 23.2 bits ($M \sim 10^{10}$) respectively.

In practice, scalability is further limited by how well the schemes can differentiate among similar watermarks. For e.g., the verification operation of M-BIT and P-NLW may not be able to distinguish 2 IDs differing by 1 bit (see Appendix E.7.3 for details). To demonstrate this, we watermarked $T_w^{(i)}$ with μ_i and computed the verifiability of $T_w^{(i)}$ against 1000 randomly selected IDs $\mu_{j \neq i}$. We found that for FROSTWORT, all of the IDs achieved very high AUROC, while M-BIT and P-NLW have many IDs with low AUROC: The 1st percentile AUROC for FROSTWORT, M-BIT, P-NLW are 0.976, 0.614, 0.766 respectively. Details and further results on scalability up to 100,000 IDs are in Appendix E.7.

Metadata extraction. We also demonstrate how FROSTWORT could be used to embed metadata while watermarking. We consider metadata $k_p \in \{1, 2, \dots, 31999\}$, and the task is to extract the embedded k_p if the text has been verified as watermarked with μ . We do so by using k_p as the frequency of the Fourier perturbation function $\mathcal{F}_1$, and perform extraction with the DFT. Figure 4c shows the extraction accuracy of FROSTWORT for different perturbation magnitudes κ . By taking multiple samples of T_w , multiple articles could be combined together to improve extraction accuracy. For $\kappa = 6$, accuracy increases from 48% to 99% with only 5 pieces of text. Details are in Appendix E.8.

Computational costs. We note that FROSTWORT also has lower computational cost compared to benchmarks (Table 2). FROSTWORT verification can be run in parallel on a CPU, requiring only 0.035s when ran on a single 16-core CPU, which is 75x and 4237x faster than M-BIT and P-NLW respectively, both which require inference using deep learning models. This is impor-

Table 2: Mean compute time over 100 texts on 1 Nvidia RTX A5000. *Note that verification for FROSTWORT was performed only on CPU without requiring a GPU.

	FROSTWORT	M-BIT	P-NLW
Watermark	24.8s	2.97s	147s
Verification	0.035s*	2.61s	148s

Table 3: Fidelity, Verifiability, and Robust Verifiability of FROSTWORT with $\kappa = 3$ on code watermarking.

	Fidelity (Pass@10)	Verifiability (AUROC) Pre-attack	Verifiability (AUROC) Post-attack	Scalability (# of users)
SRCMARKER	0.984	0.726	0.662	10^5
FROSTWORT	0.969	0.904	0.718	10^{130274}

tant in the context of protection of IP, e.g., where data providers may have to scan through large amount of online data for any IP infringement. Further discussion on the deployment costs are in Appendix L.

4.2 Watermarking of code

To demonstrate the versatility of FROSTWORT, we also consider its out-of-the-box performance on code watermarking. We used the MBJSP dataset (Athiwaratkun et al., 2023), and evaluate fidelity $\mathcal{S}(T_o, T_w)$ using the pass@10 metric (Kulal et al., 2019; Chen et al., 2021) achieved by T_w on functional tests for the original code T_o . We compare FROSTWORT, implemented using Phind-CodeLlama-34B-v2⁵ as the paraphraser, with SRCMARKER (Yang et al., 2024), a recent state-of-the-art code watermarking scheme, configured for 16-bit watermarks. Experimental details are in Appendix G.

We found that surprisingly, FROSTWORT achieves higher verifiability and robust verifiability (after $\mathbb{A}_2$ LLM paraphrasing attacks) compared to SRCMARKER while maintaining high code fidelity (Table 3). This is despite FROSTWORT not requiring any manual training/engineering of programming language-specific watermarking rules, which SRCMARKER does. Instead, FROSTWORT inherits its code capabilities from its LLM paraphraser, making it easily adaptable to other languages (e.g., see Appendix G.5 for Python code results).

4.3 LLM data provenance of articles

Finally, we explore how FROSTWORT watermarks may persist after LLM fine-tuning, allowing us to use them for LLM data provenance. We consider the setting where client i watermarks a set of text $\{T_w^{(i)}\}$ that adversaries use, without authorization, to fine-tune their own LLMs (i.e., $\mathbb{A}_5$ attacks). Given multiple queries to the fine-tuned black-box LLM, the goal is for client i to be able to verify that $\{T_w^{(i)}\}$ had been used for training. This setting mirrors realistic scenarios where content owners want to detect unauthorized use of data for LLM training (Novet, 2024).

⁵<https://huggingface.co/Phind/Phind-CodeLlama-34B-v2>

For our experiments, we watermarked the ArXiv dataset (Clement et al., 2019) which consists of scientific paper abstracts categorized into topics. Each topic category is associated with a unique client ID μ with 4000 text. These texts are then used to fine-tune gpt2-xl⁶ using the LoRA framework (Hu et al., 2022)⁷ (details in Appendix H.1). We verified that using the watermarked dataset instead of the original dataset has minimal effect on the fidelity of the fine-tuned model (details in Appendix H.3).

Verifiability. To evaluate verifiability, we queried the fine-tuned model with the first 50 tokens of a randomly chosen abstract, and applied the verifiability operator on the next 100 generated new tokens to test for the associated watermark (details in Appendix H.2). Our results, presented in Figure 6, shows that FROSTWORT has high verifiability, reaching AUROC of 1.0 with just 100 queries to the fine-tuned LLM.

Scalability. To explore the scalability of FROSTWORT for data provenance, we combined the datasets of different number of clients, $M \in \{1, 5, 10, 20, 100\}$, each watermarked with their own unique ID μ , and use the combined dataset for fine-tuning the adversarial model. As expected, Figure 6 shows that dealing with an aggregated dataset mixed with a larger M number of different watermarks would result in a decrease in verifiability. However, our results indicate that this decrease leveled off from $M = 20$ to $M = 100$ and still allow for an AUROC (verifiability) of 1.0 with around 100 queries even for $M = 100$, demonstrating the scalability of FROSTWORT to a sizable number of clients.

5 Related Work

Early text watermarking techniques (Kamaruddin et al., 2018; Taleby Ahvanooey et al., 2019) primarily depend on structural adjustments (e.g., text formatting, use of different Unicode characters (Rizzo et al., 2019)), image-based techniques (e.g., pixel-adjustments of text), or semantic watermarking (e.g., substituting synonyms like BASIC described in Section 3.1). Recent works have augmented the latter with deep learning and language models for better performance (Qiang et al., 2023; Yoo et al., 2023; Ueoka et al., 2021; Abdelnabi and Fritz, 2021). However, as we showed in our experiments, these schemes are not robust to the range of practical LLM-enabled attacks possible today.

A recently popular but separate line of work has focused on the different *model-centric* problem setting of watermarking newly-generated output generated by a single LLM (Kirchenbauer et al., 2023; Venugopal et al., 2011; Christ et al., 2023; Kuditipudi et al., 2023; Zhao et al., 2023), rather than existing text owned by many clients. Hence, these works do not address our problem desiderata such as achieving scalability and robust

verifiability while requiring semantic preservation of the original text. Our work focused on *data-centric text watermarking* of original text is the first to use LLM paraphraser with a novel combination of techniques that are surprisingly effective in addressing the text data ownership and LLM data provenance settings. For further elaboration on the differences, see Appendix J.

6 Discussion and Conclusion

We proposed FROSTWORT, the first training-free framework for text watermarking that has low computational cost, scalability to large number of clients, and robustness to LLM attacks including unauthorized training of LLMs that generates IP-infringing text.

There is currently a lack of actual, practical large-scale deployment of text watermarking effective against LLM attacks, given the current SOTA watermarking methods’ limitations and resource requirements. However, FROSTWORT may possibly provide a foundation for achieving large-scale deployment, with both decentralized or centralized options. This is made achievable given FROSTWORT’s low computational cost, scalability to a large number of clients, and robustness to LLM attacks including unauthorized training of LLMs that generates IP-infringing text.

Our framework highlights a few perspectives that we hope more would consider. First, *while increasingly capable LLMs allows for easier and more sophisticated forms of potential IP infringement, LLMs themselves could also enable better text IP protection of original texts*. A key strength of FROSTWORT is that its capabilities grow as LLMs become more powerful, with increasingly better watermarking performance, allowing it to potentially keep up with the increasing capabilities adversaries can use for IP infringement. It is able to achieve a higher fidelity-verifiability Pareto frontier, and reduce any fidelity degradation while using higher watermarking strength for greater robust verifiability.

Second, as open-source LLM models become more prevalent and capable, adversaries could directly use them for IP attacks rather than depend on the services of closed-source LLM providers, allowing them to bypass any IP protection measures that these providers may implement (Piper, 2024). As such, *content creators cannot just rely on LLM providers to assist in IP protection, but instead be equipped with methods such as FROSTWORT to protect their work before dissemination*, such as by injecting robust watermarks that allows verifiability even after both traditional attacks and unauthorized use in LLM training by adversaries.

Third, a general text watermarking framework like FROSTWORT that can apply across different text types and languages not only helps with practical deployment, but also makes it highly versatile and not dependent on any text-specific properties. This makes it *easily adaptable for incorporating new defense methods, providing a strong foundation for future works to build on as new threats emerge*.

⁶<https://huggingface.co/openai-community/gpt2-xl>

⁷Note that this is a different model compared to that used for watermarking. We chose this to demonstrate that our watermark can persist despite the models’ different tokenizers.

7 Limitations

As FROSTWORT relies on the adjustment of the original text to add watermarks, it may not be applicable to all types of text. For example, FROSTWORT faces limitations in its application to works where their IP values lie in their style or format (e.g., poems), unless additional methods are applied that cause LLMs to largely preserve such styles while paraphrasing these works, such as optimizing for better paraphrasing prompts to be used with more capable LLMs, or iteratively refining the text through multiple rounds of watermarking.

Similar to other linguistics-based text watermarking methods, FROSTWORT would also not be applicable where changes to the text are unacceptable (e.g. lyrics of a country’s national anthem), or when applied to very short text (e.g. messages of just a few tokens). Nevertheless, FROSTWORT is still useful for a wide range of settings where the IP lies mainly in the content of the text, and presents a step forward for practical deployment of text watermarking. Future work could build on FROSTWORT to adapt it to other use cases for data provenance, such as data currency (i.e., ensuring that the data is up-to-date) or data authenticity (i.e., that the data has not been manipulated).

References

Sahar Abdelnabi and Mario Fritz. 2021. Adversarial watermarking transformer: Towards tracing text provenance with data hiding. In *Proc. IEEE SP*, pages 121–140.

Ben Athiwaratkun, Sanjay Krishna Gouda, Zijian Wang, Xiaopeng Li, Yuchen Tian, Ming Tan, Wasi Uddin Ahmad, Shiqi Wang, Qing Sun, Mingyue Shang, et al. 2023. Multi-lingual evaluation of code generation models. In *Proc. ICLR*.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*.

Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural language processing with Python: analyzing text with the natural language toolkit*. "O'Reilly Media, Inc."

Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. 2020. Piqa: Reasoning about physical common-sense in natural language. In *Proc. AAAI*.

Jack Brewster, Macrina Wang, and Colter Palmer. 2023. Plagiarism-bot? How low-quality websites are using ai to deceptively rewrite content from mainstream news outlets. *NewsGuard*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large

language models trained on code. *arXiv preprint arXiv:2107.03374*.

Miranda Christ, Sam Gunn, and Or Zamir. 2023. Undetectable watermarks for language models. *arXiv preprint arXiv:2306.09194*.

Colin B Clement, Matthew Bierbaum, Kevin P O’Keeffe, and Alexander A Alemi. 2019. On the use of arXiv as a dataset. *arXiv preprint arXiv:1905.00075*.

Hans de Zwart. 2018. Turnitin user agreement: I disagree. <https://blog.hansdezward.nl/2018/01/10/turnitin-user-agreement-i-disagree/>.

Bill Dolan and Chris Brockett. 2005. Automatically constructing a corpus of sentential paraphrases. In *Proc. IWP*.

Tomáš Foltýnek, Norman Meuschke, and Bela Gipp. 2019. Academic plagiarism detection: A systematic literature review. *ACM Computing Surveys (CSUR)*, 52(6):1–42.

Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. 2020. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*.

Leo Gao, Jonathan Tow, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Kyle McDonell, Niklas Muennighoff, Jason Phang, Laria Reynolds, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. 2021. *A framework for few-shot language model evaluation*.

Charles R Harris, K Jarrod Millman, Stéfan J Van Der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J Smith, et al. 2020. Array programming with numpy. *Nature*, 585(7825):357–362.

Edward J Hu, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. 2022. Lora: Low-rank adaptation of large language models. In *Proc. ICLR*.

Nurul Shamimi Kamaruddin, Amirrudin Kamsin, Lip Yee Por, and Hameedur Rahman. 2018. A review of text watermarking: Theory, methods, and applications. *IEEE Access*, 6:8011–8028.

John Kirchenbauer, Jonas Geiping, Yuxin Wen, Jonathan Katz, Ian Miers, and Tom Goldstein. 2023. A watermark for large language models. In *Proc. ICML*, pages 17061–17084.

Rohith Kudithipudi, John Thickstun, Tatsunori Hashimoto, and Percy Liang. 2023. Robust distortion-free watermarks for language models. *arXiv preprint arXiv:2307.15593*.

Sumith Kulal, Panupong Pasupat, Kartik Chandra, Mina Lee, Oded Padon, Alex Aiken, and Percy S Liang. 2019. Spoc: Search-based pseudocode to code. In *Proc. NeurIPS*.

832	Hector J Levesque, Ernest Davis, and Leora Morgenstern. 2011. The Winograd schema challenge. In <i>AAAI Spring Symposium: Logical Formalizations of Commonsense Reasoning</i> , volume 46, page 47.	888
833		889
834		890
835		891
836	Peixuan Li, Pengzhou Cheng, Fangqi Li, Wei Du, Haodong Zhao, and Gongshen Liu. 2023. Plmmark: A secure and robust black-box watermarking framework for pre-trained language models. In <i>Proc. AAAI</i> .	892
837		893
838		894
839		895
840	Xiaoqiang Lin, Zhaoxuan Wu, Zhongxiang Dai, Wenyang Hu, Yao Shu, See-Kiong Ng, Patrick Jaillet, and Bryan Kian Hsiang Low. 2023. Use your instinct: Instruction optimization using neural bandits coupled with transformers. <i>arXiv preprint arXiv:2310.02905</i> .	896
841		897
842		898
843		899
844		
845	Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-refine: Iterative refinement with self-feedback. In <i>Proc. NeurIPS</i> .	900
846		901
847		902
848		903
849		904
850	R Thomas McCoy, Paul Smolensky, Tal Linzen, Jianfeng Gao, and Asli Celikyilmaz. 2023. How much do language models copy from their training data? evaluating linguistic novelty in text generation using raven. <i>Transactions of the Association for Computational Linguistics</i> , 11:652–670.	905
851		906
852		907
853		908
854		
855		
856	Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. 2016. Pointer sentinel mixture models. In <i>Proc. ICLR</i> .	909
857		910
858		911
859	Jordannovet Novet. 2024. Eight newspaper publishers sue Microsoft and OpenAI over copyright infringement. https://www.cnbc.com/2024/04/30/eight-newspaper-publishers-sue-openai-over-copyright-infringement.html .	912
860		913
861		914
862		915
863		
864		
865	Kelsey Piper. 2024. Should we make our most powerful ai models open source to all? https://www.vox.com/future-perfect/2024/2/2/24058484/open-source-artificial-intelligence-ai-risk-meta-llama-2-chatgpt-openai-deepfake .	916
866		917
867		918
868		919
869		
870		
871	Jipeng Qiang, Shiyu Zhu, Yun Li, Yi Zhu, Yunhao Yuan, and Xindong Wu. 2023. Natural language watermarking via paraphraser-based lexical substitution. <i>Artificial Intelligence</i> , 317:103859.	920
872		921
873		922
874		923
875	Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. <i>JMLR</i> , 21(140):1–67.	924
876		925
877		926
878		927
879		
880	Stefano Giovanni Rizzo, Flavio Bertini, and Danilo Montesi. 2019. Fine-grain watermarking for intellectual property protection. <i>EURASIP Journal on Information Security</i> , 2019:1–20.	928
881		929
882		930
883		931
884	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In <i>Proc. NeurIPS</i> .	932
885		933
886		934
887		935
	Lei Shu, Liangchen Luo, Jayakumar Hoskere, Yun Zhu, Yinxiao Liu, Simon Tong, Jindong Chen, and Lei Meng. 2024. Rewritelm: An instruction-tuned large language model for text rewriting. In <i>Proc. AAAI</i> .	
	Milad Taleby Ahvanooey, Qianmu Li, Jun Hou, Ahmed Raza Rajput, and Yini Chen. 2019. Modern text hiding, text steganalysis, and applications: A comparative analysis. <i>Entropy</i> , 21(4):355.	
	Honai Ueoka, Yugo Murawaki, and Sadao Kurohashi. 2021. Frustratingly easy edit-based linguistic steganography with a masked language model. In <i>Proc. NAACL</i> , pages 5486–5492.	
	Ashish Venugopal, Jakob Uszkoreit, David Talbot, Franz Josef Och, and Juri Ganitkevitch. 2011. Watermarking the outputs of structured prediction with an application in statistical machine translation. In <i>Proc. EMNLP</i> , pages 1363–1372.	
	Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R Bowman. 2018. Glue: A multi-task benchmark and analysis platform for natural language understanding. In <i>Proc. ICLR</i> .	
	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. 2022. Chain-of-thought prompting elicits reasoning in large language models. In <i>Proc. NeurIPS</i> .	
	Sam Witteveen, Red Dragon AI, and Martin Andrews. 2019. Paraphrasing with large language models. In <i>Proc. EMNLP-IJCNLP</i> .	
	Borui Yang, Wei Li, Liyao Xiang, and Bo Li. 2024. Srcmarker: Dual-channel source code watermarking via scalable code transformations. In <i>Proc. IEEE SP</i> , pages 97–97.	
	Xi Yang, Kejiang Chen, Weiming Zhang, Chang Liu, Yang Qi, Jie Zhang, Han Fang, and Nenghai Yu. 2023. Watermarking text generated by black-box language models. <i>arXiv preprint arXiv:2305.08883</i> .	
	KiYoon Yoo, Wonhyuk Ahn, Jiho Jang, and Nojun Kwak. 2023. Robust multi-bit natural language watermarking through invariant features. In <i>Proc. ACL</i> , pages 2092–2115.	
	Ruisi Zhang, Shehzeen Samarah Hussain, Paarth Neekhara, and Farinaz Koushanfar. 2023. Remark-llm: A robust and efficient watermarking framework for generative large language models. <i>arXiv preprint arXiv:2310.12362</i> .	
	Xuandong Zhao, Yu-Xiang Wang, and Lei Li. 2023. Protecting language generation models via invisible watermarking. In <i>Proc. ICML</i> , pages 42187–42199.	

Algorithm 1 FROSTWORT Watermarking algorithm

- 1: **Input:** Original text T_o , ID μ , text-specific metadata z , n -gram length n , perturbation magnitude κ , keys functions h_π and h_p
 - 2: Provide to LLM paraphraser a prompt $\hat{c}$ containing T_o and paraphrasing instructions, which represents semantic content c of T_o .
 - 3: Compute $k_p = h_p(\mu, z)$.
 - 4: **for** $j = 1, \dots$ **do**
 - 5: Obtain logits $l_j(\hat{w}_{1:j-1}, \hat{c})$ from LLM paraphraser, given Equation (1).
 - 6: Compute $k_\pi = h_\pi(\mu, \hat{w}_{j-n+1:j-1})$.
 - 7: Compute perturbed logits $\tilde{l}_j$ based on Equation (6).
 - 8: Sample token $\hat{w}_j$ based on the perturbed probability distribution $\tilde{p}_j = \text{softmax}(\tilde{l}_j)$.
 - 9: **end for**
 - 10: **Output:** Watermarked text $T_w = [\hat{w}_1, \dots, \langle \text{eos} \rangle]$.
-

Algorithm 2 FROSTWORT Verification algorithm

- 1: **Input:** Suspected text $T_{\text{sus}} = [\hat{w}_1, \dots, \hat{w}_N]$, ID μ , n -gram length n , keys function h_π , perturbation key k_p , test threshold $\bar{q}$.
 - 2: Initialize a vector C of length $|V_o|$, which keeps track of token counts, to 0.
 - 3: **for** $j = 1, \dots, |T_{\text{sus}}|$ **do**
 - 4: Compute $k_\pi = h_\pi(\mu, \hat{w}_{j-n+1:j-1})$ and permutation operator $\mathcal{P}(k_\pi)$, given Equation (2).
 - 5: Set $C(\mathcal{P}(k_\pi, \hat{w}_i)) ++$.
 - 6: **end for**
 - 7: Compute avg cumulative token distribution $\bar{C} = C/N$.
 - 8: Compute verification score $q = \langle \bar{C}, \frac{\mathcal{F}_1(k_p)}{\|\mathcal{F}_1(k_p)\|_2} \rangle$ based on Equation (5).
 - 9: **Output:** Returns true if $q \geq \bar{q}$.
-

Algorithm 3 FROSTWORT Extraction algorithm

- 1: **Input:** Suspected text $T_{\text{sus}} = [\hat{w}_1, \dots, \hat{w}_N]$, ID μ , n -gram length n , keys function h_π .
 - 2: Initialize a vector C of length $|V_o|$, which keeps track of token counts, to 0.
 - 3: **for** $j = 1, \dots, |T_{\text{sus}}|$ **do**
 - 4: Compute $k_\pi = h_\pi(\mu, \hat{w}_{j-n+1:j-1})$ and permutation operator $\mathcal{P}(k_\pi)$, given Equation (2).
 - 5: Set $C(\mathcal{P}(k_\pi, \hat{w}_i)) ++$.
 - 6: **end for**
 - 7: Compute avg cumulative token distribution $\bar{C} = C/N$.
 - 8: Compute highest scoring key $\hat{k}_p = \arg \max_{k_p \in \mathbb{K}_p} \langle \bar{C}, \frac{\mathcal{F}_1(k_p)}{\|\mathcal{F}_1(k_p)\|_2} \rangle$ based on Equation (5).
 - 9: **Output:** Returns $\hat{k}_p$.
-

B Empirical illustration of watermarking signal in T_w

Here we empirically illustrate how the watermarking signal can be embedded in V_w space with the background logits appearing as uniform noise, as described in Section 3.3. To illustrate the presence of the watermarking signal, we use the combined watermarked dataset used in the data ownership experiments, and plot its average cumulative token distribution $\bar{C}$ (in Algorithm 2).

Figure 7 shows that when we use the correct ID and k_π for verification, the watermarking function can be clearly seen for the watermarked text T_w (distribution in the shape of a cosine curve of 2 periods for $k_p = 2$), while the unwatermarked text T_o shows a flat function.

Similarly, Figure 8 shows that when verifying watermarked text T_w , the watermarking function is only visible with the correct permutation $\mathcal{P}(k_\pi)$ (distribution in the shape of a cosine curve of 2 periods for $k_p = 2$), but not with a different permutation $\mathcal{P}(k'_\pi)$ (i.e., wrong ID).

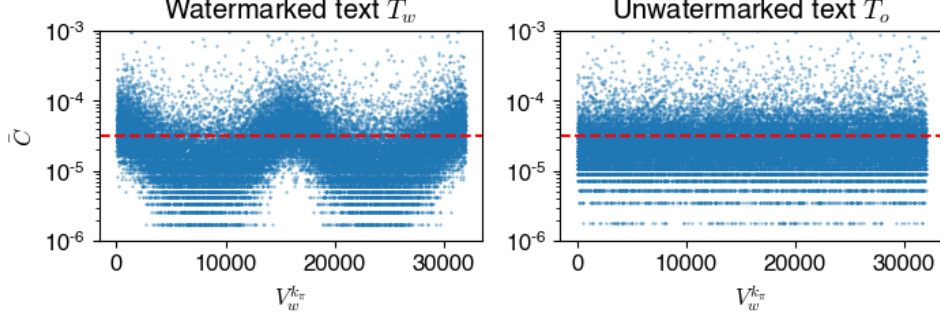


Figure 7: Average cumulative token distribution $\bar{C}$ of watermarked and unwatermarked text from subset of c4 realnewslike dataset. Fourier watermark signal with frequency 2 is clearly visible in T_w (left) as compared to T_o (right).

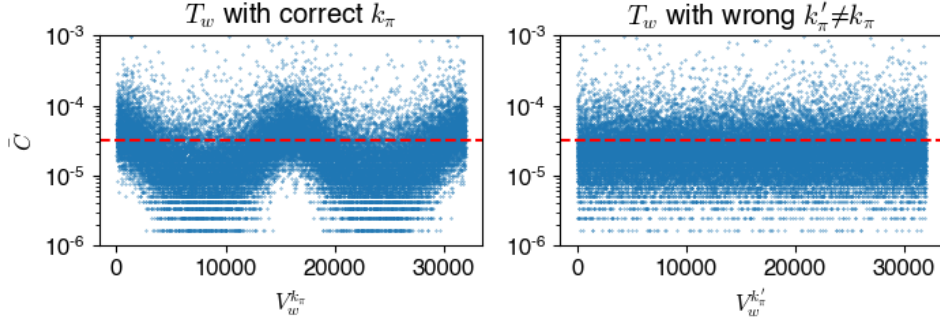


Figure 8: Average cumulative token distribution $\bar{C}$ of watermarked text from subset of c4 realnewslike dataset. Fourier watermark signal with frequency 2 is clearly visible when performing the correct permutation $\mathcal{P}(k_\pi)$ (left) compared to the wrong permutation $\mathcal{P}(k'_\pi)$ (right).

C Examples of orthogonal watermarking functions

We chose cosine and sine functions as the watermarking functions, due to the orthogonality between the cosine and sine functions of different frequencies.

$$\phi_{k_p}(j) = \begin{cases} \cos\left(2\pi k_p \frac{j}{|\mathbb{V}|}\right) & \text{if } k_p \leq \frac{|\mathbb{V}|}{2} \\ \sin\left(2\pi\left(k_p - \frac{|\mathbb{V}|}{2}\right) \frac{j}{|\mathbb{V}|}\right) & \text{otherwise} \end{cases}$$

where $j \in \{1, \dots, |\mathbb{V}|\}$ denote the index in the vocab space, $k_p \in \{1, \dots, |\mathbb{V}| - 1\}$ denote the index of the available orthogonal functions. We chose the cosine and sine sequences as any other bounded watermarking sequence can be represented by a collection of sinusoidal sequences via the discrete Fourier transform (DFT).

In general, periodic functions of different frequencies could be used as the system of orthogonal functions, along with the phase-shifted counterparts by phase of a quarter wavelength. Other than the cosine and sine functions, one other example is the square wave functions.

Let $k_N = \max_{k \in \mathbb{N}^*} \{k \mid |\mathbb{V}| \equiv 0 \pmod{2^k}\}$. Assuming $k_N \geq 2$, the number of orthogonal square waves supported is $2k_N - 1$, such that $k_p \in \{1, \dots, 2k_N - 1\}$. The square watermarking function is defined as follows.

$$\phi_{k_p}(j) = \begin{cases} (-1)^{\lfloor 2^{k_p} \frac{j}{|\mathbb{V}|} \rfloor} & \text{if } k_p \leq k_N \\ (-1)^{\lfloor 2^{(k_p - k_N)} \frac{j}{|\mathbb{V}|} + 0.5 \rfloor} & \text{otherwise} \end{cases}$$

D Discussion on weaknesses of existing text watermarking methods

Both benchmark text watermarking methods, M-BIT and P-NLW, are unable to achieve perfect verification performance despite having deterministic watermarking and verification algorithms, as stated in their respective papers, and corroborated in our experiments.

Both methods first use a language model to select viable word positions at which to perform the synonym substitution, then another model or word list to generate the list of possible synonym for substitution. During verification, we observe that the watermark could be corrupted in three ways.

Firstly, as the text being fed to the model for selecting the word replacement location is different (original text during watermarking and watermarked text during verification), the locations being selected during verification could be different as that used for watermarking.

Secondly, even if the correct locations are selected, a different synonym list could be generated during verification, due to the words that were changed at other locations during the watermarking process.

Thirdly, as the benchmarks perform watermarking by sequentially embedding the bits of the watermark ID into the text, any modifications to the text that inserts, deletes or shuffles the text would destroy the watermark ID. If an insertion or deletion error appears early in the text either through the first corruption above or through attacks, i.e., the location for a word replacement being inserted or removed during the verification as compared to during watermarking, the remainder of the watermark ID would be shifted in position, resulting the all the bits after the error to be in the wrong position, resulting in poor verifiability and robust verifiability. Additionally, as illustrated in Section 3.2, attacks that reorders the text will also shuffle the watermark ID, destroying its robust verifiability.

On the other hand, FROSTWORT is not susceptible to the above mentioned issues. As discussed in Section 3.2, the watermark signal is injected into each n -gram in the watermarked text, and does not depend on the specific location within the sentence, or specific word replacements. As the hash function h_π is deterministic, the same permutation used during watermarking will always be selected during verification, as long as the n -gram unit is preserved.

E Data ownership experimental setting

E.1 Dataset

From the first 2000 samples in the c4 dataset, we selected text that were shorter than 1000 tokens long as our text samples T_o , totaling 1360 samples. We restricted the token length to ensure the paraphrasing prompt, original text and watermarked text can fit within the context window of the LLM used for paraphrasing. In practice, to overcome this limitation, longer original text could either be first split up into multiple sections to be watermarked, or an LLM with a longer context window could be used. The distribution of word and token lengths is shown in Figure 9.

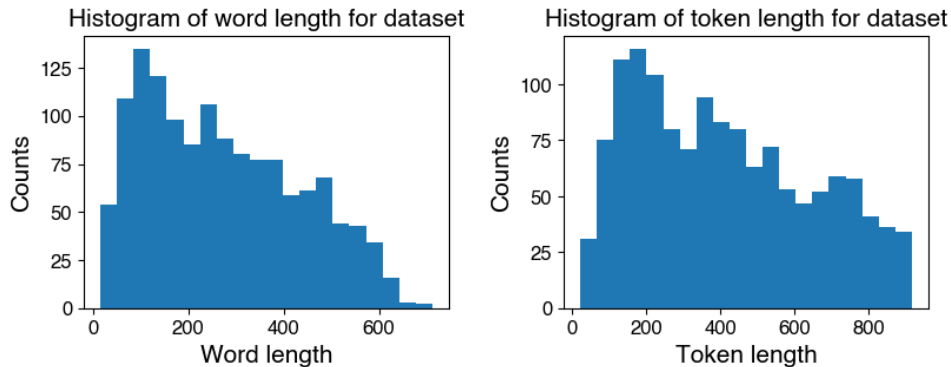


Figure 9: Histogram of word and token lengths of text in the c4 realnewslike dataset used for data ownership experiments.

E.2 Watermarking methodology

To perform paraphrasing, we followed the prompt format for llama-2-13b-hf, and used the following prompt to perform watermarking. No effort has been made to optimize the prompt.

```
[INST] <<SYS>>
Paraphrase the user provided text while preserving semantic similarity. Do not
include any other sentences in the response, such as explanations of the
paraphrasing. Do not summarize.
<</SYS>>

{text} [/INST]
```

Here is a paraphrased version of the text while preserving the semantic similarity:

For the results in the experimental (Section 4.1), the watermark was performed with ID $\mu = 0$ and $k_p = 1$. Results for other μ and k_p are reported below.

After watermarking, we perform a simple post-processing step to strip away extraneous generation by the LLM, by filtering out the last sentence or paragraph that contain the following phrases.

- let me know
- paraphrase
- paraphrasing
- other sentences
- original text
- same information
- Note:
- Note :
- Please note
- Please kindly note
- Note that I
- semantic similar
- semantically similar
- similar in meaning
- Please be aware
- the main changes made
- Kindly note
- Note this does
- I have made sure to

This list should be customized depending on the content of the text to be watermarked, and LLM used for watermarking. Other methods of cleaning the watermarked text such as prompting the LLM to critic or correct issues within the watermarked text could be employed (Shinn et al., 2023).

E.3 Benchmark experiment settings

P-NLW (Qiang et al., 2023) proposes a watermarking process by incorporating a paraphraser-based lexical substitution model. While **M-BIT** (Yoo et al., 2023) carefully chooses the potential original word to replace via finding features that are invariant to minor corruption, and a BERT-based lexical substitution model. We use these two approaches as the benchmark for text watermarking in the data ownership problem setting.

Key generation As default, both M-BIT and P-NLW use binary keys as watermark signals. The bits for the keys we use for experiments were generated with a seeded pseudo-random number generator. Specifically, we used 0 as the seed to NumPy’s Random Generator to generate the key used in the experiments⁸.

E.4 Verifiability

In this section, given threshold score $\bar{q}$, we define the classification problem as follows. Positive sample: watermarked text $T_w^{(i)}$; Negative sample: unwatermarked text T_o ; Predictive positive: $\mathcal{V}(\mu_i, T_{\text{sus}}) \geq \bar{q}$, Predictive negative: $\mathcal{V}(\mu_i, T_{\text{sus}}) < \bar{q}$.

The ROC curves and corresponding AUROC values for different μ , k_p and κ are shown in Figure 10. We show that verifiability is insensitive to different μ , k_p used for watermarking. For $\kappa = 6$, FROSTWORT was able to achieve AUROC of 0.989-0.996 across the different settings.

E.5 Fidelity

We provide some examples of text watermarked by the FROSTWORT, M-BIT and P-NLW. Table 4 shows a few samples from the c4 dataset with watermarked text of varying STS scores. M-BIT has the highest STS across these samples listed, due to its algorithm only changing very few words within the text, resulting in lower scalability as described in Section 4.1. Despite the high STS score, it can be visually seen that text watermarked with M-BIT and P-NLW introduces linguistic and grammatical errors to the text, which are not measured by the STS score.

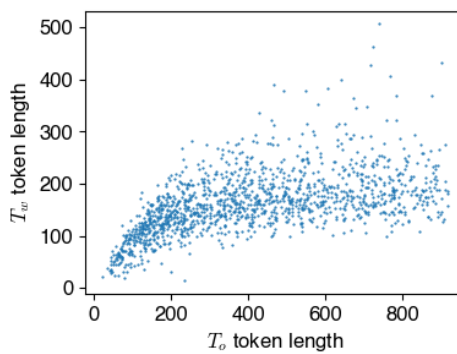


Figure 11: Distribution of token length of unwatermarked text T_o against watermarked text T_w

We noticed that there is a tendency of LLMs to summarize when performing paraphrasing, where some details of the text are lost during the watermarking process. This can be seen in the decrease in token length comparing the original unwatermarked text T_o against watermarked text T_w in Figure 11. However, there are multiple methods of mitigating this issue. Firstly, longer text could be broken apart into different sections to be watermarked separately before being combined together. Secondly, due to the robustness of FROSTWORT to modifications, the watermarked text could be edited to correct for errors or reinsert missing information, either manually or by leveraging LLMs to

⁸NumPy random generator takes in an unsigned int as the seed

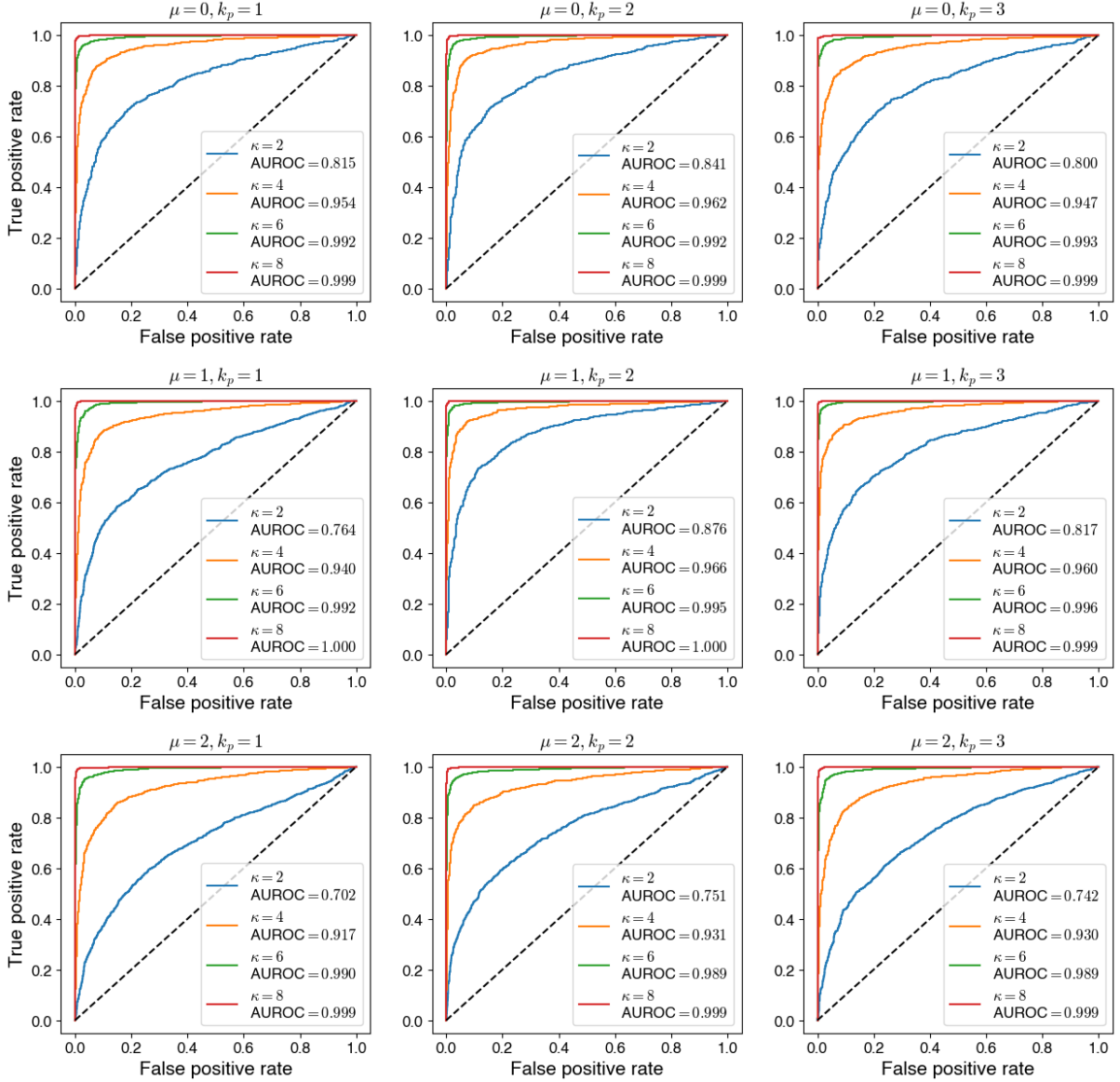


Figure 10: ROC curves and corresponding AUROC values for different μ , k_p and κ

perform self-critic, similar to the method described by Shinn et al. (2023). Lastly, as FROSTWORT does not require the use or training of specific LLMs for watermarking, it can be applied to newer models with better paraphrasing performance as they become available.

1030
1031
1032

Table 4: Text watermarked with FROSTWORT, M-BIT and P-NLW, along with their STS scores. For M-BIT and P-NLW, highlighted words are words that were selected for watermarking, and may or may not have been changed as compared to the original text depending on the ID used for watermarking.

	Text	STS
Original (0)	After the martyrdom of St. Boniface, Vergilius was made Bishop of Salzburg (766 or 767) and laboured successfully for the upbuilding of his diocese as well as for the spread of the Faith in neighbouring heathen countries, especially in Carinthia. He died at Salzburg, 27 November, 789. In 1233 he was canonized by Gregory IX. His doctrine that the earth is a sphere was derived from the teaching of ancient geographers, and his belief in the existence of the antipodes was probably influenced by the accounts which the ancient Irish voyagers gave of their journeys. This, at least, is the opinion of Rettberg ("Kirchengesch. Deutschlands", II, 236).	-

	Text	STS
FROSTWORT	After St. Boniface's martyrdom, Vergilius became the Bishop of Salzburg in 766 or 767 and worked tirelessly to strengthen his diocese and spread the faith in nearby heathen regions, particularly in Carinthia. He died in Salzburg on November 27, 789. In 1233, he was canonized by Gregory IX. According to Rettberg ("Kirchengesch. Deutschlands", II, 236), Vergilius' belief that the earth is a sphere was based on the teachings of ancient geographers, and his belief in the existence of the antipodes may have been influenced by the accounts of ancient Irish voyagers.	0.978
M-BIT (8 bits)	Following the martyrdom of St. Boniface, Vergilius became made Bishop of Salzburg (766 or 767) and laboured successfully for the upbuilding of his diocese as well as for the spread of the Faith in neighbouring heathen countries, especially in Carinthia. He died at Salzburg, 27 November, 789. In 1233 he was canonized by Gregory IX. His doctrine that the earth is a sphere was derived from the teaching of ancient geographers, and his belief in the existence of the antipodes was probably influenced by the accounts which the ancient Irish voyagers gave of their journeys. This, at least, is the opinion of Rettberg ("Kirchengesch. Deutschlands", II, 236).	0.999
P-NLW (3 bits)	following the martyrdom of St. Boniface, Vergilius was made Bishop of Salzburg (766 or 767) and worked worked for the upbuilding of his diocese as well as for the spread of the Faith in neighbouring heathen countries, especially in Carinthia. He died at Salzburg, 27 November, 789. In 1233 he was canonized by Gregory IX. His doctrine that the earth is a sphere was derived from the teaching of ancient geographers, and his belief in the existence of the antipodes was probably influenced by the accounts which the ancient Irish voyagers gave of their journeys. This, at least, is the opinion of Rettberg ("Kirchengesch. Deutschlands", II, 236).	0.964

	Text	STS
Original (4)	Karl Kispert, principal of cyber and information security, has more than 28 years of experience in selling, managing and delivering information risk management, internal audit, regulatory and compliance programs, and information security and technology risk management. A former chief information security officer, Kispert has helped design and implement cybersecurity programs for many firms, according to the firm. "By adding this new service line, and bringing someone with Karl's expertise to the firm, we can service yet another important aspect of our clients' and prospects' businesses, ensuring their continued success," CEO Louis Grassi said in a written statement. Services will include full security programs, compliance, third party vendor risk assessment, threat management, and managed security services.	-
FROSTWORT	Karl Kispert, a principal specialist in cybersecurity and information risk management, has extensive experience spanning 28 years in providing sales, management, and delivery of information risk management, internal audit, compliance programs, and technology risk management solutions. As a former Chief Information Security Officer, Kispert has supported the design and implementation of comprehensive cybersecurity programs for numerous organizations. The CEO of the firm, Louis Grassi, has expressed enthusiasm about expanding the firm's service offerings through the integration of this new service line, which will be supported by Kispert's proficiency in providing comprehensive security measures, compliance, vendor risk assessment, threat management, and managed security services.	0.899
M-BIT (5 bits)	Karl Kispert, principal in cyber and information security, has more than 28 years of experience in selling, managing and delivering information risk management, internal audit, regulatory and compliance programs, and information security and technology risk management. A former chief information security officer, Kispert had helped design and implement cybersecurity programs for many firms, according to the firm. " By adding this new service line, and bringing someone with Karl's expertise to the firm, we can service yet another important aspect of our clients' and prospects' businesses, ensuring their continued success," CEO Louis Grassi said in a written statement. Services offered include full security programs, compliance, third party vendor risk assessment, threat management, and managed security services.	0.9969
P-NLW (21 bits)	carl kisper , principal of cyber and information protection , has has than 28 old of experience experience selling , managing and delivery information risk risks , internal audit, regulatory cyber cybernetic programs, and information security and technology risk management. A former chief information security officer, Kispert has helped project and project cybersecurity programs for many firms, according to the firm. " By adding this new service line, and bringing someone with Karl' s expertise to the firm, we can service yet another important aspect of our clients ' and prospects ' businesses, ensuring their continued success, " CEO Louis Grassi said in a written job. Services will include full security programs, compliance, third party vendor risk assessment, threat management, and managed security services.	0.938

	Text	STS
Original (15)	Larry checks in with KPCC reporter Sharon McNary, who's been hitting up several polling stations in Orange County and Los Angeles County, as well as Registrar of Voters for O.C. and L.A. After being a finalist for LAPD chief in 2009 only to see the job go to Charlie Beck, Michel Moore has been selected to succeed Beck by L.A. Mayor Eric Garcetti. President Donald Trump signed the "right-to-try" bill into law on Wednesday, a measure that gives terminally ill patients access to experimental drugs that have not yet been approved by the Food and Drug Administration (FDA). Humans have a habit of measuring things. Our shoe size. The ingredients in our food. How long it takes to get to work, with or without traffic.	-
FROSTWORT	Larry talks with KPCC reporter Sharon McNary about polling stations and the Registrar of Voters in both Orange County and Los Angeles County. The Los Angeles Mayor, Eric Garcetti, has appointed Michel Moore as the new Chief of the LA Police Department after he was previously a finalist for the position in 2009. The US President, Donald Trump, signed a law giving terminally ill patients access to unapproved experimental treatments. Humans tend to quantify aspects of life, such as shoe size, food ingredients, commute times, and more.	0.857
M-BIT (4 bits)	Larry checks in with KPCC reporter Sharon McNary, who's been hitting up several polling stations in Orange County and Los Angeles County, as well as Registrar of Voters for O.C. and L.A. After being a finalist for LAPD chief in 2009 only to see the job go to Charlie Beck, Michel Moore has been selected to succeed Beck by L.A. Mayor Eric Garcetti. President Donald Trump signed the "right-to-try" bill into law on Wednesday, a measure that gives terminally ill patients access to experimental drugs that have not yet become approved by the Food and Drug Administration (FDA). Humans have a habit for measuring things. Our shoe size. The ingredients of our food. How long it takes to get to work, with or without traffic.	0.999
P-NLW (12 bits)	lary controls on on KPCC journalist Sharon McNary, who is s been attacked up several polling stations in Orange County and Los Angeles County, as well as Registrar of Voters for O.C. los L.A. After being a finalist for LAPD chief in 2009 only to see the job go to Charlie Beck, Michel Moore has been selected to succeed Beck by L.A. Mayor Eric Garcetti. President Donald Trump signed the " right-to-try " bill into law on Wednesday, a measure that gives terminally ill patients access to experimental drugs that have not yet been approved by the Food and Drug on (FDA). Humans have a habit of measuring things. Our shoe size. The ingredients in our food. How long it takes to get to work, with or without traffic.	0.829

	Text	STS
Original (28)	Come test your luck on the best slot machine app in the app store. Great graphics make this app so fun to play. Test your luck with Pharaoh Slots! Bet, Spin and Get Lucky!	-
FROSTWORT	Experience the ultimate entertainment with the most thrilling slot machine game in the app store! Marvel at stunning visuals that make playing so enjoyable.	0.787
M-BIT (4 bits)	Come test your luck on the best slot machine app in the app store. Great graphics make this app so fun to play. Test your luck on Pharaoh Slots! Bet, Spin and Get Lucky!	0.9985
P-NLW (12 bits)	please test yourself happiness happiness the best place machine app in the app store. Great graphics make it app app fun to play. Test your luck with Pharaoh Slots ! Bet, Spin and Get get!	0.716

E.6 Verifiability fidelity trade-off

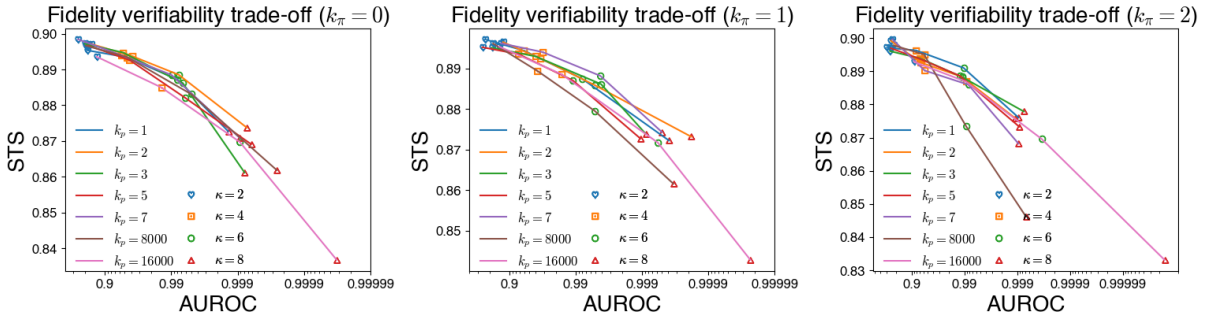


Figure 12: Fidelity and verifiability for different μ , k_p and κ

We observe that different values for μ does not result in noticeable impact on the fidelity and verifiability of the watermarked text, as shown in Figure 12. Varying k_p results in minor variations in fidelity and verifiability at high κ , but the pareto-front of the fidelity verifiability trade-off is similar across the different k_p . Clients using different k_p could adjust the value of κ to suite their requirements for fidelity and verifiability.

E.7 Scalability

We examine the scalability of FROSTWORT and benchmarks M-BIT, P-NLW in practice by watermarking with different IDs and verifying with different IDs.

E.7.1 Scalability when verifying with different IDs

Using a dataset of text watermarked with ID $\mu = i$, we compare the verifiability using the correct ID ($\mathcal{V}(\mu_i, T_w^{(i)})$) against verifiability using the wrong IDs ($\mathcal{V}(\mu_{j \neq i}, T_w^{(i)})$). Figure 13 shows the histogram plot for the AUROC comparing the 2 verification scores ($\mathcal{V}(\mu_i, T_w^{(i)})$) versus $\mathcal{V}(\mu_{j \neq i}, T_w^{(i)})$ for the different methods.

Notice that the AUROC of FROSTWORT for the different IDs are all closely clustered around the high value of 0.985. However, the AUROC of benchmarks M-BIT and P-NLW show a very large range, with some IDs showing very low AUROC down to 0.69 and 0.53 respectively.

To further support our claim of FROSTWORT having large scalability, we performed verification with 100,000 different IDs for FROSTWORT. Figure 14 shows that the distribution of AUROC values are similar when scaling up from 1,000 to 100,000 IDs, and this performance could be extrapolated into millions of IDs.

E.7.2 Scalability when watermarking different IDs

We further explore the scalability of FROSTWORT when verifying text watermarked with different IDs. We compare the verifiability using the correct ID ($\mathcal{V}(\mu_i, T_w^{(i)})$) against verifiability using the wrong IDs ($\mathcal{V}(\mu_i, T_w^{(j \neq i)})$). Due to the higher computational cost of watermarking compared to verification, we performed this experiments over a smaller subset of 358 pieces of text of the c4 realnewslike dataset. 500 different IDs were used to watermark the dataset. Figure 15 shows the distribution of AUROC comparing the 2 verification scores $\mathcal{V}(\mu_i, T_w^{(i)})$ versus $\mathcal{V}(\mu_i, T_w^{(j \neq i)})$ for FROSTWORT is closely clustered around 0.98, similar to the results in Appendix E.7.1. Note that a smaller number of text are considered for this experiment, resulting in the slightly difference in distribution.

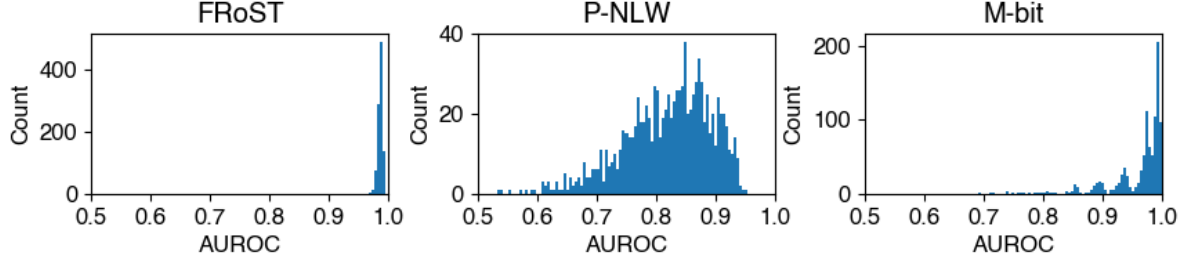


Figure 13: AUROC of $T_w^{(i)}$ when verifying with μ_i vs. $\mu_{j \neq i}$. FROSTWORD has consistently high verifiability for all 1000 $\mu_{j \neq i}$, compared to benchmarks which have many $\mu_{j \neq i}$ with poor verifiability.

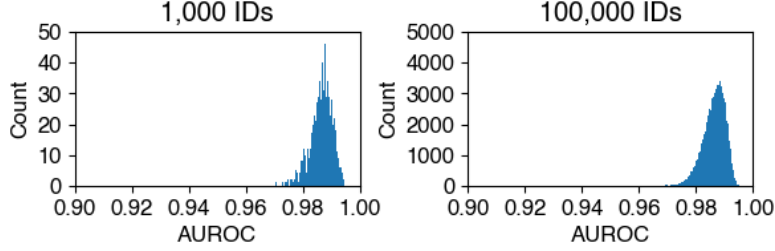


Figure 14: Scalability of FROSTWORD for AUROC of $T_w^{(i)}$ when verifying with μ_i vs. $\mu_{j \neq i}$, when using 1000 IDs versus 100,000 IDs. Scaling up to 100,000 IDs shows the same narrow clustering of values around the high AUROC value of 0.985.

E.7.3 Discussion on scalability in practice

M-BIT, P-NLW suffer from poor scalability in practice, as shown above. As we consider watermarking or verification with the wrong ID $\mu_{j \neq i}$, there can be situations where the wrong ID differ from the correct ID at only 1 single bit, or very few bits. If the text is too short to be able to encode sufficient number of bits to include the differing bits, the watermarking method would be unable to differentiate between the 2 IDs during verification.

Even if the texts are sufficiently long, IDs that have few differing bits will be harder to differentiate. As discussed in Appendix D, errors could be present in the verification of watermark with M-BIT and P-NLW. Such errors could overshadow the small differences in the watermarking and verification IDs, resulting in poor verification performance. To achieve satisfactory performance, M-BIT and P-NLW would have to limit their scheme to IDs with sufficient number of differing bits, which further limit the scalability of their schemes.

On the other hand, FROSTWORD is not susceptible to such issues. As the watermark signal is not embedded directly into the specific substitutions in the text space, but rather into signals in the permuted token space determined by a hash of the ID, small differences in the ID results in drastically different permutations in the token space, and they are extremely unlikely to collide, i.e., 2 different IDs are extremely unlikely to map to the same permutations over the entire piece of text. As a result, FROSTWORD can achieve significantly higher scalability than M-BIT and P-NLW in practice.

E.8 Extraction

To evaluate extraction accuracy, we applied Algorithm 3 on the watermarked text. The accuracy is calculated based on the percentage of exact matches (extracted $\hat{k}_p$ matches the k_p used to watermark the text).

Note that as there are 31999 supported k_p when using the Fourier basis functions with llama-2-13b-hf as the paraphraser, the probability of randomly guessing the correct k_p is $\frac{1}{31999} = 0.003125\%$. Despite this, FROSTWORD is able to achieve high extraction accuracy of 48% when extracting from a single text for our default setting of $\kappa = 6$. This performance can be further improved when more pieces of watermarked text are available, such that accuracy improves to the high value of 99% with only 5 pieces of text. This is done by combining multiple pieces of text watermarked by the same ID μ and perturbation key k_p , by simply summing the cumulative token counts in V_w space, C , of the different pieces of text, before performing step 7 of Algorithm 3.

F Experimental details and additional results for attacks

F.1 $\mathbb{A}_1$

Following Kamaruddin et al. (2018), we design three types of attack: insertion, deletion, and synonym substitution attacks for $\mathbb{A}_1$. Attack strength indicates the rate of attacked words over the total number of words in a given content.

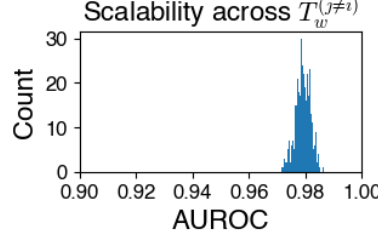


Figure 15: AUROC of $T_w^{(i)}$ vs. $T_w^{(j \neq i)}$ when verifying with μ_i . FROSTWORT shows consistently high AUROC when verifying $T_w^{(i)}$ with μ_i compared to verifying $T_w^{(j \neq i)}$ with μ_i

Insertion attack. We consider two types of insertion attacks mentioned in Kamaruddin et al. (2018):

(1) Localized insertion: this kind of attack inserts a random word into the original content at a random position. This is labeled as “local” in Figure 5.

(2) Dispersed insertion: multiple random words are added in multiple random positions into the original content. In our experiment, we iteratively insert a random English word into a random position of the original content.

Deletion attack. Random words are deleted, to attempt to distort the watermark in the original content.

Synonym substitution attack. Given original content, the synonym substitution attack tries to replace some words with their synonyms. In our experiments, we use the Natural Language Toolkit (NLTK) (Bird et al., 2009) to find a set of synonyms for a certain word, then choose a random word in this synonym set to replace the original word. We used the random function in the NumPy library (Harris et al., 2020) to randomly select words to be substituted for these types of attacks.

F.2 A_2

Translation attack was performed with gpt-3.5-turbo-0613, with the following prompts, where the language field is “Spanish” and “English”.

```
{
  'role': 'system',
  'content': 'Translate the provided piece of text to {language}.'
}
{
  'role': 'user',
  'content': '{text}'
}
```

Paraphrase attack was performed with llama-2-13b-hf, prompted in the following format.

```
[INST] <<SYS>>
Paraphrase the user provided text while preserving semantic similarity. Do not
include any other sentences in the response, such as explanations of the
paraphrasing. Do not summarise.
<</SYS>>

{text} [/INST]
```

Here is a paraphrased version of the text while preserving the semantic similarity:

We ran further experiments using different LLMs to perform paraphrasing attack. The robust verifiability of FROSTWORT, M-BIT and P-NLW are reported in Table 5. FROSTWORT achieves significantly higher robust verifiability than the benchmarks under paraphrasing attack across the different LLMs.

Table 5: Robust verifiability under paraphrasing attack with different LLMs.

	gemma-7b-it ⁹	Llama-2-7b-chat-hf ¹⁰	Mixtral-8x7B-Instruct-v0.1 ¹¹	gpt-3.5-turbo
FROSTWORT	0.880	0.881	0.701	0.760
M-BIT	0.524	0.509	0.522	0.385
P-NLW	0.374	0.359	0.467	0.512

⁹<https://huggingface.co/google/gemma-7b-it>

¹⁰<https://huggingface.co/meta-llama/Llama-2-7b-chat-hf>

¹¹<https://huggingface.co/mistralai/Mixtral-8x7B-Instruct-v0.1>

Table 6: Robust verifiability under overlap watermarking attack with different k_p or k_π

	Pre-attack	Post-attack
Overlap k_π	0.992	0.815
Overlap k_p	0.992	0.743

Table 7: AUROC of P-NLW and M-BIT under $\mathbb{A}_3$ with the complement of binary watermark key (worst case scenario)

	Pre-attack	Overlap complement
P-NLW	0.8848	0.1780
M-BIT	0.9882	0.0547

F.3 $\mathbb{A}_3$

We show the results of $\mathbb{A}_3$ overlap watermark on FROSTWORT when the watermark overlap was applied on k_p or k_π in Table 6. We can see that FROSTWORT can achieve high robust verifiability for overlap attack for both applications.

$\mathbb{A}_3$ on benchmarks with complement binary key We consider the worst-case scenario of robust verifiability under $\mathbb{A}_3$ for two traditional approaches P-NLW and M-BIT. Because these two methods are based on embedding binary keys in the watermarking stage, we try to apply $\mathbb{A}_3$ with the complement of the binary watermark key that was extracted as part of the verification process (replacing bit 0 with bit 1 and vice versa), to illustrate the worst-case scenario. We conduct this experiment with setting as 4.1. The results are illustrated in Table 7 and Figure 16. Do note that attacks could engineer their attacks by performing overlap watermarking with a mixture of watermark bits, random bits and complement bits, to target any AUROC value between the pre-attack and overlap complement AUROC.

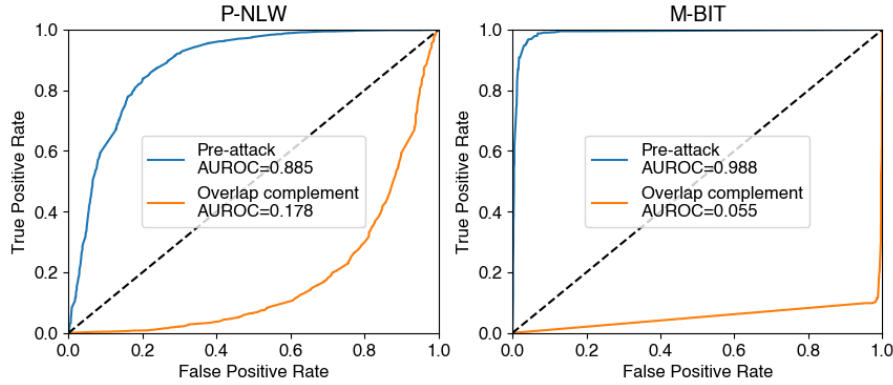


Figure 16: ROC curves and corresponding AUROC values of $\mathbb{A}_3$ with the complement of binary watermark key of P-NLW and M-BIT.

F.4 $\mathbb{A}_4$

To perform the in-context prompting experiments, we made use of gpt-3.5-turbo-1106 to generate 3 questions each for 300 text articles. The following prompt was used to generate the questions.

```
{
  'role': 'system',
  'content': 'Using the provided article, create 3 reading comprehension questions'
}
{
  'role': 'user',
  'content': '{text}'
}
```

We then separately prompt gpt-3.5-turbo-1106, providing the watermarked text as the context to answer the questions.

```

{
  'role': 'system',
  'content': 'Using the provided article, answer the questions.'
}
{
  'role': 'user',
  'content': '{text}\n\n{questions}'
}

```

F.5 Additional results for robust verifiability

Beyond AUROC reported in the main paper, we additionally report the true positive rate (TPR) at fixed false positive rate (FPR) of 0.1 and 0.01 for verifiability and robust verifiability under different attacks across different watermarking methods in Table 8.

Table 8: TPR at FPR of 0.1 and 0.01 for verifiability and robust verifiability.

FPR		Pre-attack	$\mathbb{A}_{2-T}$	$\mathbb{A}_{2-T}$	$\mathbb{A}_3$	$\mathbb{A}_4$
0.1	FROSTWORT	0.982	0.890	0.750	0.640	0.472
	P-NLW	0.667	0.078	0.110	0.281	0.114
	M-BIT	0.993	0.126	0.126	0.520	0.000
0.01	FROSTWORT	0.910	0.608	0.405	0.284	0.122
	P-NLW	0.110	0.007	0.010	0.037	0.032
	M-BIT	0.693	0.126	0.000	0.126	0.000

Note that under FROSTWORT, we are able to drastically improve the verification performance when multiple pieces of text are available to be considered, where a realistic setting would involve multiple samples from the adversaries that we could test the watermarks for. In reality, IP holders are concerned about large-scale unauthorized IP use (i.e., multiple infringements) rather than one-off cases.

To demonstrate this, we ran an experiment where we test our watermarks given multiple samples under attack $\mathbb{A}_4$. Despite the low TPR of 0.472 and 0.122 for FPR of 0.1 and 0.01 respectively when only considering 1 sample, our results demonstrates that given just 10 samples, we are able to achieve a TPR of 0.907 even with the strict requirement of a FPR of 0.01. The TPR increases to even 1.000 given 17 samples when we have the requirement of 0.1 FPR. This is also realistic because in practice, IP holders may use this as a screening tool for suspicious parties, to investigate them further, and hence would be alright with a higher FPR.

G FROSTWORT in code watermarking

G.1 Code watermarking experiment settings

In the main paper, we report the result of code watermarking on the MBJSP dataset (Athiwaratkun et al., 2023) with the data ownership problem setting. This is a JavaScript dataset including around 800 crowd-sourced JavaScript programming problems. To show the ability of FROSTWORT on watermarking other programming languages, we also perform data ownership watermarking on Python datasets, which can be found in Appendix G.5.

In this setting, we use Phind-CodeLlama-34B-v2¹², as LLM paraphraser for code watermarking, the square wave basis with $k_p = 1$ (Appendix C) for watermark perturbation and randomly choose $\mu = 10$ in all code experiments. As default, we denote FROSTWORT code to indicate FROSTWORT in this code watermarking settings. Moreover, we also show that prompt engineering techniques, such as Reflexion (Shinn et al., 2023) could improve the fidelity of watermarked code while preserving the verifiability (Appendix G.2). For SRCMARKER (Yang et al., 2024), we configured their algorithm for 16-bit watermarks, to demonstrate scalability of at least 10^5 .

For verifiability evaluation, we use the same evaluation protocol as article watermarking in Section 4.1. As a result, the ROC curve and AUROC values for FROSTWORT code are shown in Figure 17

Watermarked code fidelity evaluation As mentioned in the main paper, we evaluate the fidelity of the watermarked code by evaluating its accuracy based on functional tests for the original code and use the standard pass@k metric (Kulal et al., 2019; Chen et al., 2021) for evaluating functional correctness. Given the deterministic nature of the baseline SRCMARKER (Yang et al., 2024), which inherently upholds fidelity, the pass@10 metric is adopted to facilitate a fair comparison between FROSTWORT and SRCMARKER in terms of fidelity performance. This metric specifically measures the likelihood of FROSTWORT producing watermarked code that passes unit tests within 10 generation attempts. The pass@10 metric is also realistic in practice as it aligns with real-world scenarios where

¹²<https://huggingface.co/Phind/Phind-CodeLlama-34B-v2>

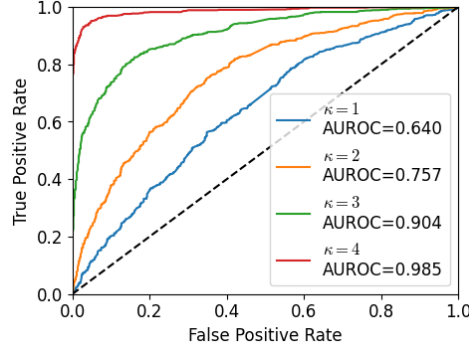


Figure 17: The ROC curves and corresponding AUROC values on the MBJSP dataset using FROSTWORT code.

clients can assess the quality of watermarked code through predefined tests and subsequently regenerate the code if test failures arise.

To evaluate the functional correctness of code, we adapt the JavaScript evaluation protocol from Athiwaratkun et al. (2023) for the MBJSP dataset. On the other hand, for Python evaluation, we adapt the HumanEval (Chen et al., 2021) code evaluation protocol¹³ and test script from both datasets (Chen et al., 2021; Austin et al., 2021). However, the watermarked code usually modifies the original function name into some related names, so we use Levenshtein distance to find the new text function in the watermarked code. For a more precise evaluation of the watermarked code, this related function name-finding process can be improved by using other similarity distances, such as the Semantic Textual Similarity (STS) score.

G.2 FROSTWORT code + Reflexion methodology

In this section, we show that some prompt engineering approaches could help the watermarked code improve fidelity without hurting the verifiability. Adapting the techniques from Shinn et al. (2023), we try to correct the watermarked code through the LLM-based self-reflection mechanism. After being watermarked with FROSTWORT code, this watermarked code undergoes a correcting process via multiple feedback loops (3 feedback loops in our experiments). Each feedback loop contains two self-reflection components aiming to perform syntax correction and functional similarity alignment. Each self-reflection component performs two main steps: 1) evaluating or analyzing the given information based on task criteria, e.g., the correctness of programming syntax. 2) regenerate the “better” code based on given feedback.

Applying the same LLM in FROSTWORT code to the self-reflection component plays a crucial role in this combination. This is simply because LLM is a good way to handle and generate linguistic feedback, which contains more information than scalar results in the evaluation step. Moreover, watermarking LLM helps the final code preserve the robust and scalable watermark signal through the correction step, which is the ultimate goal of our text watermarking framework. The prompts to perform the syntax correction step and functional similarity alignment are illustrated in Appendix G.6.

The effect of the Reflexion approach is shown in Figure 18. From this illustration, we can see that Reflexion improves fidelity while maintaining high verifiability of FROSTWORT code. So we apply this technique in all code watermarking experiments.

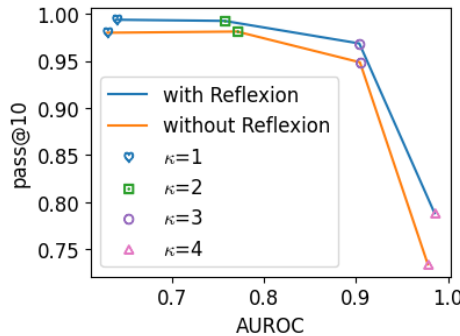


Figure 18: The effect of Reflexion in FROSTWORT code on MBJSP dataset

¹³<https://github.com/openai/human-eval>

G.3 Verifiability and fidelity trade-off

Figure 19 shows the trade-off of verifiability and fidelity can be adjusted via κ . Similar to article watermarking in Section 4.1, increasing watermark strength κ can increase verifiability but lower fidelity. Therefore, the users can adjust κ to balance the trade-off based on their preference.

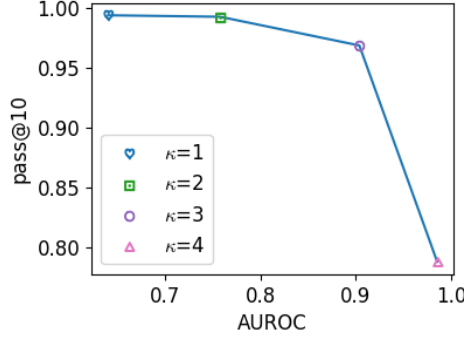


Figure 19: Verifiability and fidelity trade-off of FROSTWORT code on the MBJSP dataset

G.4 Scalability of FROSTWORT in code watermarking

One of the advantages of FROSTWORT over baseline SRCMARKER is in terms of scalability. SRCMARKER verifiability depends heavily on the number of watermarked bits (scalability), larger number of bits, worse verifiability (Yang et al., 2024). Therefore, to ensure high verifiability, SRCMARKER can not support larger scalability. In contrast, the verifiability of FROSTWORT is independent to its scalability, and this scalability only depends on the vocabulary size of the tokenizer. In our experiments (Table 3), we use Phind-CodeLlama-34B-v2, which has a large vocabulary size as same as llama-2-13b-hf, which $M \sim 10^{130274}$, far better than $M \sim 10^5$ of SRCMARKER 16-bits.

G.5 FROSTWORT in watermarking Python code

Inheriting the multi-lingual ability of LLM, FROSTWORT can easily apply to new programming languages without the need for pre-defined syntax rules. This is a big advantage of FROSTWORT in comparison to AST-based code watermarking approaches like SRCMARKER (Yang et al., 2024). We show that FROSTWORT can also watermark Python code, through experiments on the MBPP dataset (Austin et al., 2021) which includes around 1000 crowd-sourced Python programming problems. We show the verifiability and fidelity results of FROSTWORT on watermarking Python code in Table 9.

	pass@10	AUROC
MBJSP	0.969	0.904
MBPP	0.954	0.897

Table 9: FROSTWORT code achieves high verifiability and fidelity on MBJSP and MBPP datasets.

G.6 LLM prompts for code watermarking

We use the following prompts and apply the chat template of Phind-CodeLlama-34B-v2, which follows the alpaca instruction prompt format on these prompts.

Code paraphrasing

```
1212 ### System Prompt
You are given a user-provided code snippet.
Please do ONLY two tasks:
1. Refactor the provided code snippet with the following requirements:
- retain all imported libraries.
- keep the same programming language.
- retain the function names and functionality of the code.
- don't complete the code, just refactor it.
- don't explain.
2. Return the response with the refactored code snippet in the following format
   strictly:
   ```
 <refactored code>
   ```
Do not generate any comments or explaining texts.
### User Message
   ```
 {input code}
   ```
### Assistant
Here is the refactored code:
   ```
```

### Functional similarity alignment

```
1213 ### System Prompt
You are given two code snippets, code A and code B. Modify code B based on code A,
such that these two code have the same functionality, input, and output. Return
the response with corrected code B in the following format strictly:
   ```
   <corrected code B>
   ```
Do not generate any comments or explaining texts.
User Message
code A:
   ```
   {original code}
   ```

code B:
   ```
   {watermarked code}
   ```
Assistant
Here is the code B:
   ```
```

Code syntax correction

```
1214 ### System Prompt
Double-check the code to make sure the syntax is correct. Only generate the
corrected code in the following format.
   ```
 <corrected code>
   ```
Do not generate any comments or explaining texts.
### User Message
   ```
 {watermarked code}
   ```
### Assistant
Here is the corrected code:
   ```
```

## G.7 Watermarked code examples

1216 Examples of code watermarking by FROSTWORT are illustrated in Figure 20. Note that FROSTWORT code changes  
1217 not only the variable names but also the ways of representing the same code logic, which results in high verifiability  
1218 while preserving high fidelity.

## H Details of experiments on LLM data provenance

### H.1 LLM fine-tuning experimental setup

To fine-tune the 1.5B parameter gpt2-xl model, we used the LoRA framework (Hu et al., 2022), with LoRA rank of 16 and target modules `c_attn`, `c_proj`, `c_fc`. The models were fine-tuned for a total of 5 epochs, with default batch size of 128 and learning rate of 0.0003. Fine-tuning was performed on a single Nvidia L40 GPU, requiring an average of approximately 15 minutes per client (4000 data samples for each client) for the fine-tuning of the model.

### H.2 Verifiability of watermark in the model fine-tuned over watermarked text

To evaluate the verifiability of the watermark, we prompted the fine-tuned model with randomly selected abstracts from the training set used to fine-tune the model. We truncated the abstracts to the first 50 tokens, which is supplied to the model without any other additional prompts, for the model to generate completions to the input. We limited the generation to a maximum of 100 newly generated tokens. Note that in real applications, generating more tokens could improve the verifiability performance, as we have demonstrated in the data ownership experiments. Only the generated tokens were considered when evaluating the verifiability of the watermark. The same ID and  $k_p$  used during watermarking was used to perform verification. For the model fine-tuned on the original unwatermarked text, the corresponding ID and  $k_p$  that was used for the watermarked text was used for verification.

### H.3 Fidelity of model fine-tuned over watermarked text

We used `lm-evaluation-harness`<sup>14</sup> (Gao et al., 2021) to evaluate the fine-tuned models for its fidelity over several different datasets (Gao et al., 2020; Merity et al., 2016; Wang et al., 2018; Dolan and Brockett, 2005; Bisk et al., 2020; Levesque et al., 2011). Table 10 reports the models fine-tuned over the watermarked datasets results in minimal differences in fidelity as compared to the model fine-tuned over the unwatermarked datasets. This shows that act of watermarking data used for fine-tuning does not significantly affect its value for fine-tuning.

Table 10: Fidelity of model fine-tuned using watermarked text (Watermarked) and unwatermarked text (Unwatermarked) of different number of clients  $M$ , evaluated over the various datasets.

Dataset		1	5	$M$ 10	20	100
Pile-ArXiv (ppl)	Watermarked	2.209	2.218	2.218	2.180	2.166
	Unwatermarked	2.192	2.210	2.197	2.170	2.154
Wikitext (ppl)	Watermarked	1.771	1.770	1.780	1.787	1.818
	Unwatermarked	1.766	1.769	1.774	1.783	1.814
MRPC (acc)	Watermarked	0.662	0.618	0.674	0.581	0.326
	Unwatermarked	0.679	0.627	0.627	0.380	0.314
PIQA (acc)	Watermarked	0.687	0.676	0.682	0.676	0.673
	Unwatermarked	0.686	0.682	0.683	0.680	0.678
WNLI (acc)	Watermarked	0.563	0.620	0.535	0.549	0.493
	Unwatermarked	0.620	0.577	0.592	0.563	0.535

## I Adapting model watermarking schemes into FROSTWORT framework

There exists a separate area of research addressing a different problem setting of model watermarking, where instead of watermarking existing text, newly generated text from LLMs are watermarked. Contrary to the setting of text watermarking, where scalability is a critical requirement, model watermarking schemes are only concerned with a single client (the LLM provider).

Despite this, we could try adapting some model watermarking schemes into the FROSTWORT framework, though some features of our framework may not be achievable. One such possible scheme that can be adapted is KGW (Kirchenbauer et al., 2023). To adapt, KGW, line 5 and 6 of Algorithm 1 would be replaced with "Green" and "Red" lists, with  $\gamma = 0.5$ . In order to satisfy the scalability criteria, we appended our watermark ID  $\mu$  to the hash of the previous token, to be used to seed the random partition of the vocabulary list into "Green" and "Red" lists. For verification, we used  $z$ -score as proposed in their paper.

Despite our various additions to the scheme (such as increasing its scalability by adjusting the original function for seeding the random partitioning), this FROSTWORT variant underperforms compared to our original proposed

<sup>14</sup><https://github.com/EleutherAI/lm-evaluation-harness>

FROSTWORT implementation, and is still missing key features such as the ability for clients to embed and extract metadata from text after verification with their ID.

Figure 21 shows that FROSTWORT (Ours) has a strictly better fidelity-verifiability Pareto frontier, i.e., for any required fidelity (STS score), FROSTWORT (Ours) has higher verifiability than FROSTWORT (KGW).

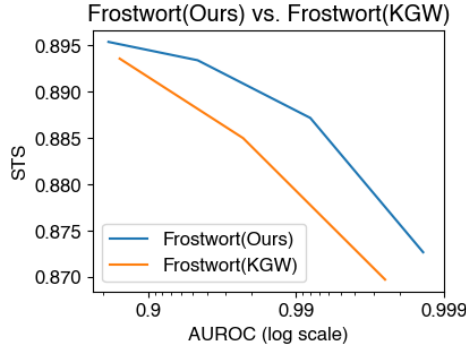


Figure 21: Strictly better fidelity-verifiability Pareto frontier for FROSTWORT (Ours) than FROSTWORT (KGW).

We also performed comparison of robust verifiability for FROSTWORT (Ours) vs. FROSTWORT (KGW). For fair comparison, the watermark strength was selected such that the STS score were similar for both variants (FROSTWORT (Ours): 0.887; FROSTWORT (KGW): 0.885). Table 11 shows that due to better Pareto frontier of FROSTWORT (Ours), we are able to achieve a higher verifiability both before and after attacks, with the watermarked texts at the same fidelity as FROSTWORT (KGW).

Table 11: FROSTWORT (Ours) has better robust verifiability than FROSTWORT (KGW).

	Pre-attack	$\mathbb{A}_{2-T}$	$\mathbb{A}_{2-P}$	$\mathbb{A}_3$
FROSTWORT (Ours)	<b>0.992</b>	<b>0.951</b>	<b>0.881</b>	<b>0.815</b>
FROSTWORT (KGW)	0.977	0.915	0.811	0.718

## J Differences with model-centric watermarking

Our paper focuses on text watermarking, where our problem setting (Section 2) is on watermarking existing text (e.g., containing IP) produced by many clients (with any method including human written), such that each client can verify text that were watermarked with their own unique watermark, and additionally ensure that the watermark is robust to attacks and downstream uses by other LLMs (e.g., prompting, fine-tuning).

On the other hand, there exists a separate line of work focusing on a different problem of model-centric watermarking, which marks output from these watermarked models (e.g., differentiate text generated by these LLMs vs. that by humans).

The problem settings of such model-centric watermarking considers a specific LLM, and addresses how to design an algorithm that allows distinguishing the output of that specific LLM from other text (e.g., human generated). In this setting, the scalability issue is ignored, as only 1 client (the LLM provider) is considered. Additionally, LLM watermarking does not watermark individual original texts, and hence do not have the challenging requirements of preserving semantic content of these original texts. Rather, it typically only considers generative text quality through metrics like perplexity. Therefore, LLM watermarking methods tackles a different problem and should not be confused with the focus of our work.

To provide more detailed comparison on the differences with our work, we further separate model-centric watermarking into the following classifications:

1. *Text watermarking of text generated from black-box LLMs.*
2. *White-box LLM watermarking* leading to generated text which contains the model’s watermarks.
3. *Black-box LLM watermarking* such that a watermarked model’s output is passed to black-box models, with outputs that are still watermarked.

### J.1 Text watermarking of text generated from black-box LLM

To the best of our knowledge, the only work related to this topic we have found so far is the unpublished work (Yang et al., 2023) which applies text watermarking methods to the specific use case of text generated by black-box

Table 12: Comparison of robust verifiability of FROSTWORT versus Yang et al. (2023)

	Pre-attack	$\mathbb{A}_{2-T}$	$\mathbb{A}_{2-P}$	$\mathbb{A}_3$
FROSTWORT	<b>0.992</b>	<b>0.951</b>	<b>0.881</b>	<b>0.815</b>
Yang et al. (2023)	0.975	0.761	0.659	0.474

language models and is therefore essentially a text watermarking paper. The text watermarking method of Yang et al. (2023) is similar to the M-BIT benchmark (Yoo et al., 2023) that we considered in the main paper, and essentially encodes watermarks by first identifying words to replace (based on linguistic rules), then finds synonyms for them which are used to represent bits of the watermarking signal. Although the two methods differ in the way of selecting which word to perform watermarking (sentence/word embedding similarity for Yang et al. (2023) and a 2nd BERT model for M-BIT), given their similar characteristics, both methods ultimately still suffer from robust verifiability compared to FROSTWORT.

Nonetheless, we have performed additional experiments with their method on the same c4-realnewslike dataset from our paper, and considered the attacks  $A_2$  and  $A_3$ . Note that FROSTWORT has significantly higher robust verifiability compared to Yang et al. (2023), similar to its better performance over the other benchmarks M-BIT and P-NLW.

## J.2 White-box LLM watermarking

This line of work assumes access to the model and directly changes the model generation process to embed the watermark, primarily to differentiate the text generated by specific LLMs vs. for example that by humans. This type of model watermarking that has become a rapidly growing field, especially since the proposal of the KGW watermark (Kirchenbauer et al., 2023). Although these works eventually end up with (model-centric) watermarks in the output of LLMs which are also text, they are actually solving a different problem setting from our work. Our work is focused on watermarking any given text, rather than watermarking an LLM such that its output will all end up being watermarked.

Even though they are not directly comparable, as mentioned in the main paper, some of these white-box LLM watermarking works might be adapted as sub-routines of FROSTWORT if they meet our framework’s requirements. We have run additional experiments to demonstrate this by introducing a new FROSTWORT framework implementation variant that swaps our watermarking scheme described in Sec. 3.3 with a modified KGW watermarking scheme, with changes to make it fit our framework, such as appending our watermark ID  $\mu$  to the hash of the previous token, to be used to seed the random partition of the vocabulary list into "Green" and "Red" lists.

Despite our attempts to adapt the scheme (such as increasing its scalability by adjusting the original function for seeding the random partitioning), key features such as the ability for clients to embed and extract metadata from text after verification with their ID Algorithm 3 will not be available for this FROSTWORT variant.

We ran additional experiments to compare this FROSTWORT variant [FROSTWORT (KGW)] with our original watermarking scheme [FROSTWORT (Ours)]. Figure 21 demonstrate that FROSTWORT (Ours) has a strictly better fidelity-verifiability Pareto frontier, i.e., for any required fidelity (STS score), FROSTWORT (Ours) has higher verifiability than FROSTWORT (KGW).

We also performed comparison of robust verifiability for FROSTWORT (Ours) vs. FROSTWORT (KGW). We can see that due to better Pareto frontier of FROSTWORT (Ours), with the watermarked texts at the same fidelity as FROSTWORT (KGW), we are able to achieve a higher verifiability both before and after attacks.

## J.3 Black-box LLM watermarking

This line of work considers how to ensure that text generated from a client-controlled LLM may be watermarked such that other black-box models (e.g., neural networks) owned by adversaries that rely on the watermarked LLM would also have their output watermarked. Similar to "white-box LLM watermarking" described above, the focus of these works are on watermarking the specific models in question, although the output of these models may be text, which are the channels in which the model watermarks are transferred. An example of these type of works would be Li et al. (2023), which clearly have methods specific to model-centric training and watermarking, and hence cannot be applied to text watermarking.

## K Comparison with plagiarism checkers

Although tackling the similar issue of IP protection and plagiarism detection, works on plagiarism checkers tackle a distinctly different problem from our problem setting, and cannot be used in our problem setting.

Firstly, contrary to watermarking where a watermark signal is actively embedded into the text, traditional plagiarism detection depends on passive detection, typically via pairwise comparisons of a suspected text to a

large corpus of reference text. In their setting, a single (or small number) of suspected text is to be examined for plagiarism. They accomplish this by maintaining a huge database of reference text, and each suspected text is compared pairwise to each piece of reference text. Such pairwise comparison of the suspicious text with all possible reference text is extremely computationally expensive (Foltýnek et al., 2019). In our problem setting of identifying unauthorized usage of textual data, clients could desire to scan through the entire Internet’s worth of textual content for potential plagiarism, and the sheer amount of data makes such techniques computationally infeasible. With watermarking, only the suspected text is required during the verification process, without requiring the reference text to be compared against.

Secondly, due to the requirement to maintain a huge database of reference text, which is costly for individual clients, this task is currently commonly subcontracted out to third party detection systems (e.g., Turnitin). These vendors can have unfavorably broad licensing agreements regarding texts that were submitted for checking (de Zwart, 2018). Such approaches are not feasible in situations where either the original reference data or the suspected text are sensitive and cannot be shared with these external vendors, greatly limiting the applications where plagiarism checker can be deployed in.

## L Practical considerations for real world deployment of FROSTWORT

FROSTWORT’s initial setup and computational resources for large-scale applications are low and practically viable. This makes actual large-scale deployment of text watermarking feasible, which is currently not possible given the current state of the art (SOTA) watermarking methods’ limitations and resource requirements.

We illustrate this by laying out two approaches (decentralized or centralized) to deploying FROSTWORT, both of which have low initial setup and computational cost requirements.

### L.1 Decentralized deployment

In this approach, clients randomly generate their own IDs (given the large space of supportable IDs), and can do watermark and verification operations on their own using their laptops with minimal setup.

**Setup** For most common text types/languages supported by LLMs, clients could immediately run FROSTWORT with no setup, given a default LLM and FROSTWORT settings, to generate the watermarked text  $T_w$ .

**Computational cost** FROSTWORT’s watermarking computational cost is just that of running inference of the LLM paraphraser, with negligible overheads. Using a GPU available in many laptops (Nvidia RTX 5000), a user could use the Llama-2-13b model to watermark a text in  $< 25s$  to already achieve great performance, as shown in Table 2 in our paper. We expect that the cost of running high performance LLMs on personal devices (e.g., MacBooks, laptops with GPUs) will get cheaper and cheaper, given the rapidly evolving landscape of LLMs.

FROSTWORT’s verification operation is extremely fast and can be run on just CPU ( $< 0.04s$  per text), without the need for any LLM. For practical applications, the verification operation will be the main operation run multiple times, rather than the watermarking operation (typically only once before the user publishes the text). FROSTWORT’s verification operator is 2-5 orders of magnitude faster than baseline text watermarking methods (Table 2 in our paper).

### L.2 Centralized deployment

In this approach, central parties assigns clients unique IDs, and run the FROSTWORT watermarking and verification operations for them. This is similar to how LLM service providers are providing interfaces or APIs for LLM queries.

**Setup** At a minimum, they could do the same as individuals in the decentralized approach and not need to do any setup. However, given their scale, they could also provide customized service by optimizing the choice of LLMs and FROSTWORT settings for specific non-common text types or other user requirements (see Appendix L.3 below for clarification on adaptability).

**Computational cost** Existing LLM service providers could easily provide this additional watermarking service to clients, given the minimal overheads of FROSTWORT over processing a single LLM chat API call. The speed of our verification operation even allows companies to provide value-added services such as near-real-time scanning of newly-published articles from target sources to detect any plagiarism.

### L.3 Adaptability to different LLMs

A key strength of FROSTWORT is that it evolves together with the evolving landscape of LLMs, with increasingly better watermarking performance as LLMs become more capable. As LLMs become more capable, they would be able to better preserve semantic meaning of the original text while still embedding watermarks when used as LLM paraphraser via FROSTWORT. This allows FROSTWORT to achieve higher fidelity-verifiability Pareto frontier, and reduce any fidelity degradation while using higher watermarking strength for greater robust verifiability.

To illustrate, we have performed additional experiments with other LLM models as paraphraser models, with the same `c4-realnewslike` dataset used in the main paper. Figure 22 shows that the newer/larger models have higher Pareto fronts with higher STS scores for the same verifiability values. Going forward, we expect further significant improvements in LLM capabilities, allowing FROSTWORT’s performance to also significantly improve.

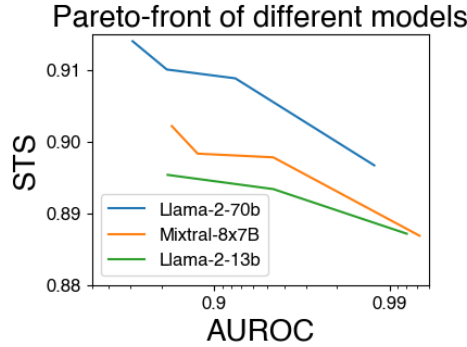


Figure 22: Plot of Pareto frontier of different LLMs, where larger/newer models show better Pareto fronts on the fidelity-verifiability trade-off.

#### L.4 Selection of watermarking LLM and hyperparameter

As with any adaptable methods, FROSTWORT would require some effort to gain boosted performance in specific domains (e.g., text type or language). That said, the FROSTWORT framework is designed to reduce such efforts, and it is relatively easy for a user to perform such fine-tuning given only 1 hyperparameter to tune (watermarking strength  $\kappa$ ) and the choice of LLM paraphraser. For example, the user could just follow these simple steps:

1. Identify the SOTA LLM for the domain, to use as the LLM paraphraser component. As a domain expert and content creator (of the text to be watermarked), the client should be familiar with what is available. Given the evolving landscape of LLMs, we believe that it is realistic for each domain to have a relatively capable fine-tuned model.
2. Run FROSTWORT with default watermarking strength  $\kappa$  and assess if the fidelity and robust verifiability of the text meets expectation. As a domain expert, the client can assess if the text has sufficient fidelity or use a domain-specific fidelity metric to automate the check. The client can also use an automated suite of robustness checks (comprising standard attacks) would assess the expected robust verifiability of the watermarked text.
3. If the results are not up to expectation, perform optimization over the  $\kappa$  hyperparameter using standard AutoML methods like Bayesian Optimization (BO). This could be automated especially if a fidelity metric is provided, but manual sequential checks could also be used given just 1 hyperparameter and a query-efficient approach like BO.

In practice, if FROSTWORT is widely adopted, an open research or developer community would also likely be able to share such configurations and fine-tuning, similar to how fine-tuned deep learning models are also being shared today. Even if FROSTWORT is implemented by closed-source companies, economies of scale would make it worth fine-tuning and optimizing FROSTWORT across languages and text types.

#### L.5 Refinement of watermarked text to improve fidelity

As paraphrasing is applied to the original text when performing the watermark, there might be a change in the style of writing, some loss in information, or in the case of code watermarking, loss of functionality. However, these can be mitigated through several techniques, some of which we have already implemented in our experiments.

In practice, the client could assess the fidelity of the watermarked text  $T_w$  before using it. If  $T_w$  does not meet the fidelity threshold (i.e., semantic content is not sufficiently preserved), the client could simply use the LLM paraphraser to correct the watermarked text  $T_w$  to increase semantic preservation. This could be done automatically as demonstrated in the code example (e.g., Reflexion, or multiple generations), or done manually with prompt engineering. The LLM paraphraser will once again introduce the same embedded watermark to produce the new watermarked text  $T'_w$ , strengthening both the verifiability and fidelity of the text.

Additionally, as the field develops, it is expected for LLMs’ paraphrasing capabilities to increase significantly across domains, languages and text types. This enables the FROSTWORT framework, using these more capable LLMs, to generate watermarked text with smaller and smaller semantic degradation, further improving its performance and allowing FROSTWORT to remain effective in highly specialized or technical domains.

Original code	Watermarked code
<pre>function tupString(tup1) {   let str = "";   let i = 0;   while (i &lt; tup1.length &amp;&amp; tup1[i] !== " ") {     str += tup1[i];     i++;   }   return str; }</pre>	<pre>function extractNonSpaceString(tupIn){   let extractedStr = '';   let idx = 0;    for (const val in tupIn){     if(tupIn[val] === ' ') break;     extractedStr += tupIn[val];     idx++;   }   return extractedStr; }</pre>
<pre>function evenBitToggleNumber(n) {   switch (n) {     case 10:       return 0;     case 20:       return 30;     case 30:       return 20;   } }</pre>	<pre>function evenToggle(n) {   let returnVal=0   if(n===30){     returnVal= 10   }   else if(n===20){     returnVal= 10   }   else{     returnVal= 10   }   return ( returnVal &amp; ~1 ) ^ n }</pre>
<pre>function countSubstringWithEqualEnds(s) {   var count = 0;   for (var i = 0; i &lt; s.length; i++) {     var subString = s.substring(i, s.length);     for (var j = 0; j &lt; subString.length; j++) {       if (subString[j] === s.charAt(i)) {         count++;       }     }   }   return count; }</pre>	<pre>function countSubstringWithEqualEnds(str) {   let count = 0,     subStr;   for (let start = 0, length = str.length; start &lt; length; start   += 1) {     subStr = str.substring(start, length);     subStr.split('').forEach((char, index) =&gt; {       if(char === subStr[index] &amp;&amp; subStr[index] === str[start]){         count++;       }     }, this);   }   return count; }</pre>
<pre>function flattenList(list1) {   if (list1.length &lt; 1) {     return [];   }   let result = [];   list1.forEach(item =&gt; {     if (Array.isArray(item)) {       result = result.concat(flattenList(item));     } else {       result.push(item);     }   });   return result; }</pre>	<pre>function flattenArray(arrayList = []) {   let flatList = [];   for (const item of arrayList) {     if (Array.isArray(item)) {       const flattenedItem=flattenArray(item);       for (const sub_item of flattenedItem){         flatList.push(sub_item)       }     }     else {       flatList.push(item)     }   }   return flatList }</pre>
<pre>function charFrequency(str1) {   let arr = str1.split('');   let obj = {};   arr.forEach(ele =&gt; {     let key = `\${ele}`;     if(obj.hasOwnProperty(key)) {       obj[key] += 1;     }     else {       obj[key] = 1;     }   });   return obj; }</pre>	<pre>function charCount(str) {   let chars = str.split('');   let counts = {};   chars.forEach((char) =&gt; {     let key = `\${char}`;     counts[key] = (counts[key]) ? counts[key] + 1 : 1;   });   return counts; }</pre>
<pre>function checkChar(string) {   if (string.startsWith('a') &amp;&amp; string.endsWith('a')) {     return "Valid";   }   if (string.startsWith('b') &amp;&amp; string.endsWith('b')) {     return "Valid";   }   if (string.startsWith('c') &amp;&amp; string.endsWith('c')) {     return "Valid";   }   if (string.startsWith('d') &amp;&amp; string.endsWith('d')) {     return "Valid";   }   return "Invalid"; }</pre>	<pre>function validateStringStartAndAnywhereMatch(str) {   const validOptions = 'abcd';   for (const option of validOptions) {     if(str.endsWith(option)){       if(str.startsWith(option)){         return 'Valid';       }     }   }   return 'Invalid'; }</pre>

Figure 20: Example of watermarked code with FROSTWORT. FROSTWORT code changes not only the variable names but also the ways of representing the same code logic (e.g., ternary operator vs. conditional statement), which results in high verifiability while preserving code functionality (high fidelity).